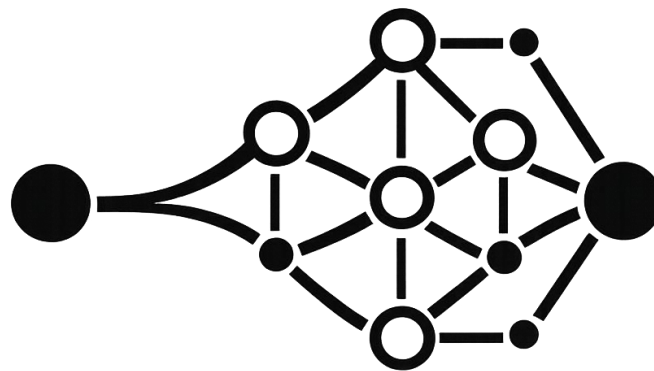


D.T.D: Data to Deployment





Ain Shams University
Faculty of Computer & Information Sciences
New Programs for Higher Education
Artificial Intelligence Program

D.T.D: Data To Deployment (Agent-Based framework for AutoML)

**This documentation is submitted in partial fulfillment of the requirements for the
Bachelor's degree in Computer and Information Sciences**

BY

Raghad Rafat Abdelkhalek Mahmoud Ahmed Mahgoub	[AI Department]
Haneen Akram Ahmed Mohamed Farid	[AI Department]
Reem Ahmed Ismail Ali	[AI Department]
Zeina Shawkat Mohamed Nabil	[AI Department]
Aly Hany Mohamed Fathy	[AI Department]
Mohamed Ashraf Fathy Eltelbany	[AI Department]

Under Supervision of

Assoc. Prof. Dina Khattab

Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

TA. Mohammad Essam

Bioinformatics Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

TA. Manar Sultan

Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

June 2026

Acknowledgements

All praise and thanks are due to Allah, who granted us the strength, patience, and perseverance to complete this work. We pray that this effort is accepted and that it serves as a beneficial contribution to the field.

We would like to express our deepest gratitude to our parents, families, and loved ones for their unwavering support, encouragement, and belief in us throughout our academic journey. Their patience, understanding, and continuous motivation were invaluable, especially during the long nights, demanding deadlines, and challenging phases of this project. Their sacrifices and support provided us with the determination needed to reach this milestone, and we are profoundly grateful for everything they have done for us.

We would also like to extend our sincere appreciation to our supervisors, Dr. Dina Khattab from the Scientific Computing Department, Faculty of Computer and Information Sciences, Ain Shams University; T.A. Mohammad Essam from the Bioinformatics Department, Faculty of Computer and Information Sciences, Ain Shams University; and T.A. Manar Sultan from the Scientific Computing Department, Faculty of Computer and Information Sciences, Ain Shams University. Their guidance, expertise, constructive feedback, and continuous support played a vital role in shaping this work and overcoming the challenges encountered throughout the project. Their dedication and encouragement have greatly enriched both our research experience and professional growth.

Finally, we would like to thank our friends and colleagues who stood by us throughout this journey. Their encouragement, assistance, and understanding, particularly during periods of intensive work and late-night development sessions, helped us maintain our motivation and successfully complete this project. We are truly grateful to everyone who contributed, directly or indirectly, to making this achievement possible.

Abstract

This paper proposes the development of **Data-to-Deployment (DTD)**, an intelligent platform that integrates Generative AI, Large Language Models (LLMs), and multi-agent frameworks such as LangGraph to automate the complete data science workflow. DTD orchestrates a network of specialized agents to handle exploratory data analysis, data preprocessing, feature engineering, model training, and deployment. The system features a dual-mode execution strategy: a fully automated static mode for generating rapid baseline pipelines, and an interactive dynamic mode that incorporates human-in-the-loop feedback to adapt to user preferences. The system is designed to process raw data, generate insights both before and after preprocessing, and construct an end-to-end AutoML pipeline that selects suitable models through systematic analysis and evaluation. By leveraging established libraries such as Pandas for data processing alongside advanced AutoML techniques, the platform streamlines preprocessing, encoding, feature engineering, and model training. The system recommends default pipeline configurations and models to ensure strong baseline performance, while users retain the flexibility to modify steps such as encoding choices, preprocessing strategies, or model selection. A central feature is the integration of LLM-powered agents that guide users through the pipeline, explain decisions in natural language, and adapt dynamically to new datasets. This design makes the workflow interactive, transparent, and adaptable to varying user expertise. The result is a platform that automates both data analysis and ML pipelines, balancing automation with user control. By unifying preprocessing, AutoML, and deployment within a single framework, the solution reduces manual effort, accelerates experimentation, and makes advanced analytics broadly accessible.

Table of Contents

Acknowledgements	i
Abstract.....	ii
Table of Contents	iii
List of Figures.....	vi
List of Tables	vii
List of Abbreviations	viii
List of Equations	x
Chapter 1 Introduction.....	2
1.1 Problem Definition	2
1.2 Motivation.....	3
1.3 Objectives	3
1.4 Time Plan	5
1.5 Document Outline.....	6
Chapter 2 Background	8
2.1 Introduction.....	8
2.2 Foundations of AutoML	8
2.2.1 Automated ML (AutoML).....	8
2.2.2 Agent-Based Systems	9
2.3 Scientific Background	9
2.3.1 ML Lifecycle	9
2.3.2 Large Language Models (LLMs)	11
2.3.3 Multi-Agent System Architectures.....	11
2.3.4 Workflow Orchestration and State-Based Execution.....	13
2.4 Literature Review	15
2.4.1 Traditional AutoML Frameworks	15
2.4.2 Automated Data Analysis Systems	16
2.4.3 LLM-Based Data Science Systems	17
2.4.4 Agent-Based AutoML Systems.....	18
2.5 Commonly Used Technologies.....	20
Chapter 3 Proposed System	24
3.1 System Overview	24

3.1.1 System Architecture	24
3.1.2 Functional Requirements.....	26
3.1.3 Nonfunctional Requirements.....	26
3.1.4 System Users	27
3.2 System Analysis & Design	29
3.2.1 Use Case Diagram	29
3.2.2 Class Diagram	33
3.2.3 Sequence Diagrams	36
3.2.4 Database Diagram	42
Chapter 4 System Implementation and Testing.....	47
4.1 Datasets	47
4.1.1 Titanic Dataset (Binary Classification)	47
4.1.2 Iris Dataset (Multi-Class Classification)	48
4.1.3 Wine Quality Dataset (Multi-Class Classification).....	49
4.1.4 Generic Dataset Upload (CSV Ingestion)	50
4.2 Modules	51
4.2.1 Intent Detection	51
4.2.2 EDA (Exploratory Data Analysis)	53
4.2.3 Preprocessing.....	54
4.2.4 Feature Engineering.....	58
4.2.5 AutoML	59
4.2.6 Knowledge Graph.....	63
4.2.7 HITL (Human-in-the-Loop)	65
4.2.8 Web Application.....	67
4.3 Results and Discussion	70
4.3.1 Experimental Environment and Setup.....	70
4.3.2 Module Verification Benchmarks	71

4.3.3 Dataset Performance Comparison Table and Discussion	74
Chapter 5 User Manual	77
5.1 Installation Guide.....	77
5.2 User Guidelines.....	83
Chapter 6 Conclusions and Future Work.....	99
6.1 Conclusions.....	99
6.2 Future Work	101
6.2.1 Compute and Performance Optimization	101
6.2.2 Functional Feature Extensions	101
Tools.....	103
References	105

List of Figures

Figure 1.1 Time Plan	5
Figure 2.1 ML Life Cycle	10
Figure 3.1: D.T.D System Architecture	25
Figure 3.2: D.T.D Use Case Diagram	32
Figure 3.3: D.T.D Class Diagram	35
Figure 3.4: D.T.D Static Pipeline Sequence Diagram	38
Figure 3.5: D.T.D Dynamic Pipeline Sequence Diagram	41
Figure 3.6: MongoDB ERD Database Schema Diagram	43
Figure 4.1: Intent Detection Processing Diagram	52
Figure 4.2: EDA Profiling and Statistical Summary Diagram	54
Figure 4.3: Preprocessing Sequential Transformation Diagram	56
Figure 4.4: Preprocessing Sequence Flowchart	58
Figure 4.5: Feature Proposal and Selection Diagram	59
Figure 4.6: AutoML Tuning and Stacked Ensembling Diagram	60
Figure 4.7: Knowledge Graph Metadata Contract Pipeline	63
Figure 4.8: Knowledge Graph Construction	64
Figure 4.9: Human-in-the-Loop Control Flow Diagram	65
Figure 4.10: Three-Tier Web Application Architecture Diagram	67
Figure 5.1: Login Page	83
Figure 5.2: Signup Page	84
Figure 5.3: Forgot Password Page	85
Figure 5.4: Home Page	86
Figure 5.5: Reach Us Option	87
Figure 5.6: Create Report Page	88
Figure 5.7: Dynamic Workflow Selection	89
Figure 5.8: Uploading Dataset	90
Figure 5.9: Target Selection Option	91
Figure 5.10: AI Target Suggestions	92
Figure 5.11: Static Workflow Nodes	93
Figure 5.12: Dynamic Workflow Knowledge Graph	93
Figure 5.13: Accept and Continue agent actions	94
Figure 5.14: Enter Feedback Prompt	95
Figure 5.15: All User Reports Page	96
Figure 5.16: Report Options	96
Figure 5.17: Visualizations and reporting	97

List of Tables

Table 2.1: Literature Review Summary Table	19
Table 3.1: MongoDB Collections and their roles in the D.T.D System.....	44
Table 3.2: Core One-to-Many relationships in the D.T.D System.....	45
Table 4.1: Dataset Characteristics Summary.....	50
Table 4.2: Feature-Specific Preprocessing Mapping (Titanic Case Study)	55
Table 4.3: Hyperparameter Tuning Ranges & Parameter Types	61
Table 4.4: Knowledge Graph Node Categories & Color Mappings	64
Table 4.5: Core API Endpoint Mapping Router Contracts	68
Table 4.6:Dataset Performance Comparison Table.....	74
Table 6.1: List of Tools	103

List of Abbreviations

Abbreviation	Stands For
ADA	Automated Data Analysis
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
AutoML	Automated Machine Learning
CBR	Case-Based Reasoning
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
D.T.D	Data To Deployment
EDA	Exploratory Data Analysis
ERD	Entity Relationship Diagram
GBM	Gradient Boosting Model
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HITL	Human-in-the-loop
HPO	Hyperparameter Optimization
HTTP	Hypertext Transfer Protocol
IQR	Interquartile Range
JSON	JavaScript Object Notation
JWT	JSON Web Token
KG	Knowledge Graph
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
MAS	Multi-Agent System
ML	Machine Learning
MSE	Mean Squared Error
NLP	Natural Language Processing
ODM	Object-Document Mapper
ONNX	Open Neural Network Exchange

RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SSE	Server-Sent Events
TPE	Tree-structured Parzen Estimator
TPOT	Tree-based Pipeline Optimization Tool
UI	User Interface
VRAM	Video Random Access Memory
XAI	Explainable Artificial Intelligence

List of Equations

Equation 4.1: Mean Equation	53
Equation 4.2: Variance Equation.....	53
Equation 4.3: Skewness Equation	53
Equation 4.4: Median Equation	56
Equation 4.5: Mode Equation.....	56
Equation 4.6: Target Encoding Equation	56
Equation 4.7: Standard Scaling Equation.....	57
Equation 4.8: Min-Max Scaling Equation.....	57
Equation 4.9: Robust Scaling Equation.....	57
Equation 4.10: IQR Boundary Clipping Equation	57
Equation 4.11: Z-score Threshold Equation.....	57
Equation 4.12: SMOTE Equation.....	57
Equation 4.13: Class Weights Equation	58
Equation 4.14: Expected Improvement Equation.....	60



Chapter One

Introduction

Chapter 1 Introduction

1.1 Problem Definition

The development of Machine Learning (ML) solutions remains a complex and time-consuming process that requires expertise across multiple stages of the ML lifecycle. Before a model can be trained, practitioners must perform a series of tasks, including data exploration, data cleaning, feature engineering, preprocessing, model selection, hyperparameter optimization (HPO), evaluation, and deployment. Each stage often requires specialized knowledge and manual intervention, creating significant barriers for users who lack extensive experience in ML.

Although Automated ML (AutoML) frameworks have emerged to simplify model development, most existing solutions focus primarily on automating model selection and hyperparameter tuning. Critical stages such as exploratory data analysis (EDA), preprocessing strategy selection, data quality assessment, feature engineering, and interpretability often remain partially manual or require substantial user expertise. As a result, users still need to understand complex ML concepts to effectively utilize these systems.

Another limitation of existing AutoML solutions is the lack of transparency in their decision-making processes. Many frameworks operate as black-box systems that generate results without providing clear explanations regarding preprocessing decisions, model choices, or optimization strategies. This limits user trust and makes it difficult to understand, validate, and improve the generated ML pipelines.

Furthermore, current solutions often provide either complete automation with limited customization or extensive configuration options that can overwhelm non-technical users. Consequently, there is a need for a system that balances intelligent automation with user-guided customization while maintaining transparency throughout the ML workflow.

This project addresses these challenges through the development of D.T.D (Data To Deployment), an agent-based AutoML framework that automates the end-to-end ML process. The proposed system integrates Large Language Models (LLMs), AI agents, and autoML techniques to support dataset analysis, preprocessing, model selection, evaluation,

and deployment. The framework aims to reduce manual effort, improve explainability, accelerate experimentation, and make advanced ML capabilities accessible to both novice and expert users.

1.2 Motivation

Developing ML solutions still requires significant manual effort in selecting algorithms, engineering features, and tuning hyperparameters. While existing AutoML tools automate parts of this workflow, they remain fragmented and often demand expert knowledge to configure and interpret. This creates long experimentation cycles and steep learning curves for most developers, and especially for users without prior ML experience. Meanwhile, the space of potential model architectures, preprocessing techniques, and parameter combinations has grown exponentially, making manual exploration both impractical and time-consuming, diverting effort away from innovation and deployment.

Managing data further intensifies these challenges. Traditional data exploration methods are slow, error-prone, and resource-intensive, ultimately delaying the extraction of actionable insights and hindering timely decision-making. While existing AutoML frameworks have attempted to simplify parts of this workflow, they are limited in two major ways: They typically optimize model selection and tuning, while leaving critical steps like data cleaning, feature extraction, and interpretability largely manual. They operate as black boxes, providing results without clearly explaining why certain choices were made.

To overcome these bottlenecks, our project proposes a fully automated end-to-end ML pipeline, extending beyond data analysis to encompass dataset ingestion, model development, and deployment. The system aims to reduce the technical burden on users, streamline data exploration, and accelerate experimentation. This enables developers to focus on higher-level innovation while empowering non-expert users to rapidly generate accurate, high-performing solutions without requiring deep ML expertise.

1.3 Objectives

The primary objective of this project is to design and develop an adaptive ML system enhanced by LLMs and AI agents to support both novice and expert users in building ML

solutions. The system seeks to balance automation with prompt-driven customization, while ensuring adaptability across diverse datasets.

The aim of the project:

1. AutoML Pipeline: Provide a default end-to-end workflow, including data preprocessing, feature engineering, model selection, and evaluation, that serves non-technical users seeking quick and reliable results, while also offering experts a baseline pipeline that can be further customized or refined.

2. AI Agent Integration: Coordinates and manages the pipeline, handling tasks such as dataset analysis, preprocessing, model selection, and debugging while adapting to user feedback [1].

3. Dataset Analysis and Reporting: Generate automatic reports [2] and visualizations from the uploaded dataset to give users clear EDA and initial insights without requiring prior technical expertise. The system will generate analysis for both raw and clean data.

4. Adaptive Preprocessing: support both novice and expert users, offering automated preprocessing for quick results, while allowing experienced users to customize preprocessing strategies directly through the graphical user interface (GUI).

5. Automated and Guided Model Selection: Select the most suitable models and pipelines based on dataset characteristics and user-defined preferences, while also allowing prompt-driven adjustments.

6. Knowledge Graph (KG) Construction: a graph that maps the entire pipeline, including preprocessing steps, model selection, and evaluation, to enhance explainability and give users the option to request modifications.

7. Self-Debugging and Recommendations: Implement self-debugging mechanisms that can detect issues within the pipeline and propose fixes, while also integrating user feedback prompts for guided improvement.

8. Prompt-Driven User Control: Allow users to provide feedback and specify preferences through natural language prompts at each stage of the pipeline. The system interprets these prompts to refine workflows, adjust preprocessing methods, select algorithms, or tune hyperparameters, thereby combining automation with user-guided customization [3].

1.4 Time Plan

Gantt chart showing the schedule of research, planning, development, integration, testing, deployment, and documentation activities throughout the project lifecycle.

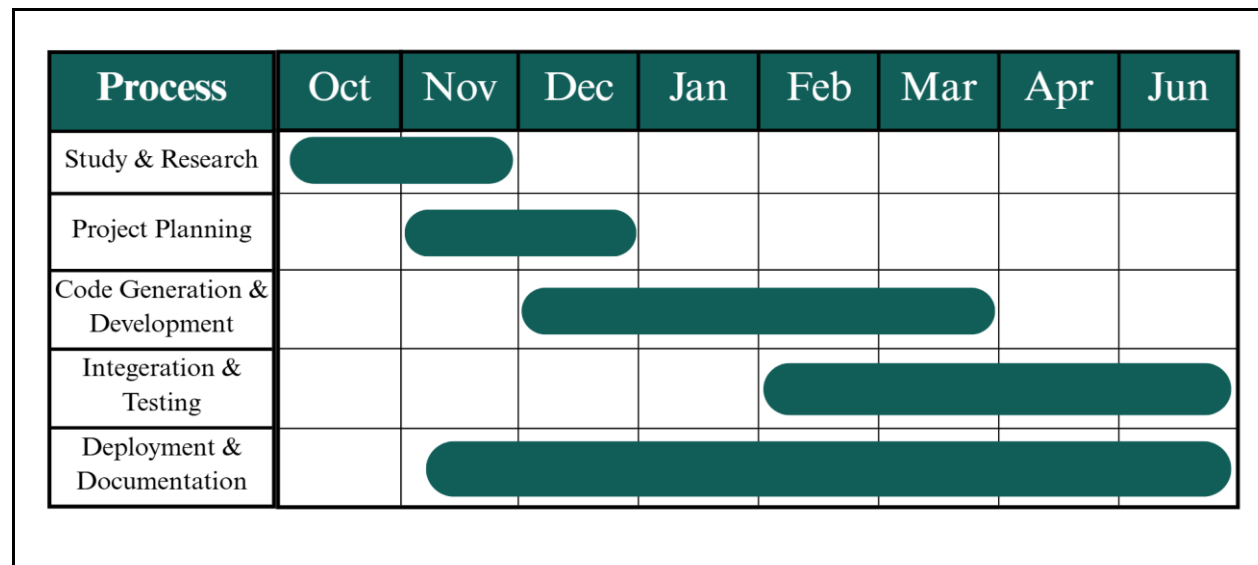


Figure 1.1 Time Plan

1.5 Document Outline

This document is organized into six chapters, each addressing a specific aspect of the D.T.D framework.

Chapter 1 introduces the project and presents its motivation, problem definition, objectives, and overall structure. It provides the context for the research and highlights the challenges addressed by the proposed solution.

Chapter 2 presents the theoretical and scientific background relevant to the project. It discusses the ML lifecycle, AutoML, LLMs, multi-agent systems (MASs), knowledge graphs, and related work in the field.

Chapter 3 describes the analysis and design of the proposed framework. It presents the overall system architecture, functional and non-functional requirements, system users, use case diagrams, class diagrams, sequence diagrams, and other design specifications necessary for understanding the structure of the system.

Chapter 4 discusses the implementation of the framework and explains the techniques, algorithms, and technologies employed during development. It provides detailed descriptions of the EDA Agent, Preprocessing Agent, Modeling Agent, and other supporting components that constitute the autoML pipeline.

Chapter 5 serves as the user manual for the system. It explains how users can operate the platform, interact with its features, and configure its components. The chapter also includes installation instructions, software requirements, and deployment guidelines.

Chapter 6 concludes the document by summarizing the project's achievements, discussing the results obtained, identifying current limitations, and outlining potential directions for future development and research.



Chapter Two

Background

Chapter 2 Background

2.1 Introduction

This chapter presents the theoretical foundations and related work relevant to the proposed D.T.D framework. It begins by introducing the primary research fields upon which the framework is built, followed by the scientific concepts required to understand its architecture. Subsequently, a review of existing research and similar systems is presented to identify current limitations and highlight the research gap addressed by the proposed solution.

2.2 Foundations of AutoML

The proposed D.T.D framework lies at the intersection of AutoML and intelligent agent-based systems. These fields aim to reduce human involvement in complex computational tasks by automating decision-making processes and workflow execution.

2.2.1 Automated ML (AutoML)

AutoML aims to automate the development of ML models by reducing the need for extensive human expertise and manual intervention. Traditional ML workflows require practitioners to perform tasks such as data preprocessing, feature engineering, algorithm selection, HPO, model evaluation, and deployment. These activities are often time-consuming and require significant domain knowledge [4].

AutoML addresses these challenges by automating key stages of the ML pipeline. Common capabilities include automated feature processing, model selection, HPO, and ensemble generation. Popular frameworks such as Auto-sklearn [5], TPOT [6], H2O AutoML [7], and AutoGluon [8] have demonstrated competitive performance across benchmark datasets [9]. However, most AutoML systems primarily focus on model optimization and provide limited support for data understanding, intelligent preprocessing decisions, explainability,

and deployment preparation [4]. These limitations have motivated research into more intelligent automation approaches that integrate reasoning and decision-making capabilities.

2.2.2 Agent-Based Systems

Agent-based automation is an emerging AI paradigm that employs autonomous agents capable of perceiving information, reasoning about tasks, and performing actions to achieve specific objectives. Recent advances in LLMs have significantly enhanced agent capabilities, enabling them to interpret instructions, generate plans, utilize tools, and make decisions based on intermediate results [10].

A common extension of this concept is the Multi-Agent System (MAS), where multiple specialized agents collaborate to solve complex problems. In ML environments, different agents may independently perform tasks such as data analysis, preprocessing, model selection, evaluation, and deployment while coordinating toward a shared goal [11]. Recent systems, including DS-Agent [10], Autonomous Data Agents [11], and AutoM3L [12] demonstrate how agent-based architectures can automate broader portions of the ML workflow. By combining AutoML techniques with intelligent reasoning, agent-based systems provide a promising foundation for end-to-end ML automation, forming the basis of the proposed D.T.D framework.

2.3 Scientific Background

2.3.1 ML Lifecycle

The ML (ML) lifecycle refers to the sequence of stages involved in developing, deploying, and maintaining ML solutions. Although implementations vary across applications, the lifecycle generally includes the following stages:

- **Data Collection:** Gathering data from sources such as databases, sensors, Application Programming Interface (APIs), or user-generated content.

- **EDA:** Understanding dataset characteristics through descriptive statistics, visualization, missing-value analysis, outlier detection, and relationship analysis.
- **Data Preprocessing and Feature Engineering:** Improving data quality through cleaning, transformation, encoding, scaling, feature creation, and feature selection [13].
- **Model Training and Evaluation:** Training ML algorithms and assessing performance using metrics such as accuracy, precision, recall, F1-score, or RMSE.
- **Deployment and Monitoring:** Integrating the selected model into a production environment and continuously monitoring performance to detect degradation and support retraining when necessary.

Despite this structured workflow, many stages still require substantial human expertise and manual effort. Tasks such as data analysis, preprocessing decisions, model selection, and deployment preparation remain challenging to automate fully. These limitations have motivated the development of AutoML and agent-based systems that seek to automate larger portions of the ML lifecycle.

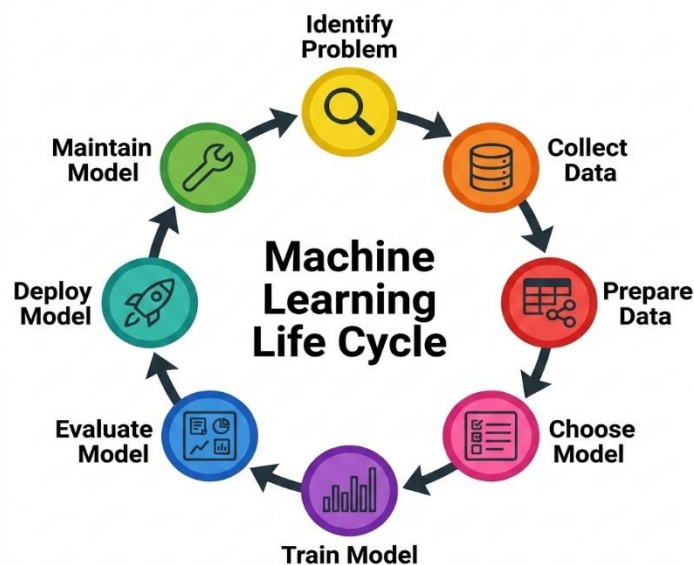


Figure 2.1 ML Life Cycle

2.3.2 Large Language Models (LLMs)

LLMs are AI systems trained on large-scale textual datasets to understand and generate natural language. Beyond traditional natural language processing (NLP) tasks, modern LLMs can perform reasoning, code generation, information extraction, and problem-solving activities, making them useful for a wide range of applications [10].

Recent developments have enabled LLMs to interact with external tools such as databases, code execution environments, and information retrieval systems. Through these capabilities, LLMs can support practical analytical workflows rather than functioning solely as text-generation systems. Examples include Data Interpreter [14], which uses LLM-generated code for data analysis, and DS-Agent [10] and AutoM3L [12], which leverage LLMs to automate data science and ML tasks.

Despite their strengths, LLMs may produce inaccurate information and are limited by the knowledge available during training [15]. Consequently, modern systems often combine LLMs with external tools and retrieval mechanisms to improve reliability. Their reasoning and tool-use capabilities make them a key component of intelligent automation systems, including the proposed D.T.D framework.

2.3.3 Multi-Agent System Architectures

A MAS is an artificial intelligence (AI) architecture composed of multiple autonomous agents that collaborate to achieve a common objective. Unlike traditional centralized systems that rely on a single decision-making component, MASs distribute tasks among specialized agents, each responsible for a specific function within the overall workflow. Through communication, coordination, and information sharing, agents can collectively solve complex problems more efficiently than a single agent operating independently.

The concept of MASs originated from distributed AI research, where complex tasks were decomposed into smaller, manageable subtasks that could be executed by independent

entities. Each agent possesses a degree of autonomy, allowing it to perceive its environment, reason about available information, make decisions, and perform actions without direct human intervention. This distributed approach improves scalability, flexibility, and maintainability while enabling systems to adapt to dynamic environments.

Recent advances in LLMs have significantly expanded the capabilities of intelligent agents. Modern LLM-powered agents can understand natural language instructions, perform reasoning, generate executable code, retrieve information from external sources, and utilize specialized tools to accomplish complex objectives [10]. As a result, multi-agent architectures have become increasingly popular for automating sophisticated workflows in domains such as software engineering, scientific research, business process automation, and data science.

A typical multi-agent architecture consists of several specialized agents that cooperate through a shared communication mechanism. Agents may exchange information directly through messages or indirectly through a shared memory structure that stores intermediate outputs and decisions. Coordination mechanisms determine how agents interact, when tasks are assigned, and how dependencies between different stages of execution are managed. This modular design enables individual agents to be developed, tested, and improved independently without affecting the overall system architecture.

In ML applications, MASs allow different agents to specialize in distinct stages of the ML lifecycle. For example, one agent may perform EDA, while other agents handle preprocessing, feature engineering, model selection, training, evaluation, or deployment. By distributing responsibilities among specialized agents, the system can achieve higher levels of automation while maintaining transparency and explainability throughout the workflow [11].

Recent research has demonstrated the effectiveness of agent-based approaches for ML automation. Systems such as DS-Agent utilize multiple reasoning components to automate data science tasks through case-based reasoning (CBR) and tool usage [10]. Similarly,

Autonomous Data Agents employ specialized agents that collaborate to analyze datasets and support data-driven decision making [11]. AutoM3L further demonstrates how multiple intelligent agents can coordinate ML activities while leveraging the reasoning capabilities of LLMs [12].

The proposed D.T.D framework adopts a multi-agent architecture to automate the end-to-end ML lifecycle. Specialized agents are responsible for tasks such as data analysis, preprocessing, model selection, training, evaluation, and deployment. Through coordinated execution and shared state management, the framework aims to combine the automation capabilities of AutoML with the reasoning and decision-making capabilities of intelligent agents, providing a transparent and scalable solution for ML development.

2.3.4 Workflow Orchestration and State-Based Execution

Workflow orchestration refers to the automated coordination and management of tasks within a complex process. In intelligent systems, orchestration mechanisms determine the order in which tasks are executed, manage dependencies between components, and ensure that information flows correctly throughout the workflow. By organizing individual operations into a structured sequence, workflow orchestration improves efficiency, consistency, and scalability.

Traditional software systems often implement workflows through static procedural logic, where execution paths are predefined and difficult to modify. While such approaches are effective for simple applications, they become increasingly difficult to manage as workflows grow in complexity. ML and AI systems frequently involve multiple interconnected tasks that require dynamic decision-making and adaptive execution strategies. Consequently, more flexible orchestration approaches are needed to coordinate these processes efficiently.

One widely adopted approach is graph-based workflow orchestration [1]. In this model, tasks are represented as nodes within a directed graph, while edges define execution paths and dependencies between tasks. Graph-based workflows provide a clear representation of

complex processes and support advanced capabilities such as conditional routing, parallel execution, checkpointing, and dynamic task selection based on intermediate results.

State management plays a critical role in orchestrated workflows. A shared state acts as a centralized repository that stores information generated during execution. Individual tasks can read from and write to this state, allowing components to communicate and coordinate without requiring direct dependencies on one another. This approach improves modularity, simplifies maintenance, and facilitates information sharing across different stages of a workflow.

Modern intelligent systems increasingly combine workflow orchestration with autonomous agents. In such architectures, specialized agents perform individual tasks while relying on a shared state to exchange information and coordinate decisions. Outputs generated by one agent can influence the behavior of downstream agents, enabling adaptive execution based on the current context. Workflow orchestration mechanisms also support conditional execution paths, allowing systems to dynamically select appropriate actions according to intermediate outcomes.

Human-in-the-Loop (HITL) mechanisms are frequently integrated into orchestrated workflows to provide oversight and control [3]. Rather than fully automating all decisions, HITL systems allow users to review intermediate results, provide feedback, and approve critical actions before execution continues. This approach improves reliability, increases transparency, and enables collaboration between automated systems and human users.

In ML automation, workflow orchestration facilitates the coordination of tasks such as EDA, preprocessing, feature engineering, model selection, training, evaluation, and deployment. By managing dependencies and information flow between these stages, orchestration frameworks enable intelligent systems to automate large portions of the ML lifecycle while maintaining flexibility, transparency, and user control. These principles form a fundamental component of the proposed D.T.D framework, which employs orchestrated agent execution and shared state management to automate end-to-end ML workflows.

2.4 Literature Review

The field of ML automation has evolved significantly over the past decade. Early research primarily focused on automating model selection and HPO, leading to the emergence of AutoML frameworks. More recently, advances in LLMs and intelligent agents have expanded automation capabilities beyond model development to encompass broader data science workflows. This section reviews the major categories of systems relevant to the proposed D.T.D framework and analyzes their strengths and limitations.

2.4.1 Traditional AutoML Frameworks

Traditional AutoML frameworks were developed to reduce the complexity of ML model development by automating tasks such as algorithm selection, HPO, and pipeline construction. These systems aim to improve accessibility while maintaining competitive predictive performance across various ML tasks [4].

One of the earliest and most influential AutoML frameworks is Auto-Sklearn, proposed by Feurer *et al.* [5]. The framework extends the Scikit-Learn ecosystem by combining Bayesian optimization, meta-learning, and ensemble construction techniques. Auto-Sklearn automatically searches for suitable ML pipelines and has demonstrated strong performance across numerous benchmark datasets while requiring minimal user intervention.

TPOT, introduced by Olson and Moore [6], approaches AutoML from a different perspective by utilizing genetic programming. The framework automatically generates and optimizes ML pipelines through evolutionary search, exploring combinations of preprocessing methods, feature selection techniques, and ML algorithms. This approach enables the discovery of effective pipelines without requiring extensive manual experimentation.

H2O AutoML provides a scalable industrial AutoML platform that automates model training, HPO and ensemble generation [7]. The framework emphasizes usability and

scalability, making it suitable for enterprise applications involving large datasets and diverse ML tasks.

AutoGluon further extends automation capabilities by providing a user-friendly framework capable of automated model selection, hyperparameter tuning, and ensemble construction [8]. Experimental evaluations have demonstrated competitive predictive performance while significantly reducing the effort required to develop ML solutions.

Despite their effectiveness, traditional AutoML frameworks primarily focus on model optimization stages. EDA, intelligent preprocessing decisions, workflow orchestration, explainability, and deployment preparation often remain partially manual or require substantial user expertise [4]. Consequently, researchers have explored more advanced approaches that automate larger portions of the ML lifecycle.

2.4.2 Automated Data Analysis Systems

As the importance of data understanding became increasingly apparent, researchers and practitioners developed ADA systems to assist users in exploring datasets and identifying quality issues before model development.

Tools such as Pandas Profiling automate EDA by generating comprehensive reports containing descriptive statistics, missing-value analysis, distribution summaries, correlation matrices, and data quality assessments [2]. Similar platforms, including Sweetviz and DataPrep, provide visual and statistical summaries that reduce the effort required to understand dataset characteristics.

While these systems significantly improve data exploration efficiency, they primarily function as reporting tools. They generate insights and visualizations but typically do not make autonomous decisions regarding preprocessing strategies, feature engineering, or model development. As a result, users must still interpret generated reports and manually determine appropriate actions.

Recent research has attempted to extend automation beyond static reporting. Data Interpreter, proposed by Hong *et al.* [14], utilizes LLMs to analyze datasets, generate code, execute analytical tasks, and produce natural-language explanations. This approach

demonstrates how intelligent systems can move beyond descriptive reporting toward more autonomous analytical workflows.

Although ADA systems reduce the effort required for dataset exploration, most remain focused on isolated stages of the data science process rather than providing comprehensive end-to-end ML automation.

2.4.3 LLM-Based Data Science Systems

The emergence of LLMs has introduced new possibilities for automating data science workflows. Unlike traditional automation tools that rely on predefined rules and search strategies, LLM-based systems can perform reasoning, generate code, interpret intermediate results, and adapt their behavior to different analytical tasks.

Data Interpreter represents one of the most prominent examples of this paradigm [14]. The system combines natural language understanding with code generation capabilities, enabling users to perform complex data analysis tasks through conversational interactions. By automatically generating and executing code, the system can conduct exploratory analysis, create visualizations, and derive insights from datasets with limited user guidance. LLM-powered systems provide several advantages over traditional automation approaches. Their ability to understand user intent, reason about intermediate outputs, and dynamically adapt workflows enables more flexible problem-solving. Furthermore, natural-language explanations improve transparency and accessibility for users with varying levels of technical expertise.

However, LLM-based systems also present important challenges. Generated outputs may be inconsistent, non-deterministic, or occasionally inaccurate. Additionally, many systems focus primarily on assisting users during analysis rather than fully automating the ML lifecycle. As a result, researchers have explored combining LLM reasoning capabilities with specialized autonomous agents.

2.4.4 Agent-Based AutoML Systems

Recent research has increasingly focused on agent-based architectures that combine autonomous decision-making, tool use, and machine-learning automation. These systems represent a significant advancement over traditional AutoML frameworks by incorporating reasoning and workflow-orchestration capabilities.

DS-Agent, proposed by Guo *et al.* [10], empowers LLMs with CBR and tool-use capabilities to automate data science tasks. The system demonstrates how intelligent agents can analyze datasets, formulate solutions, execute analytical procedures, and iteratively refine results. Experimental results indicate that agent-based reasoning can improve the quality and effectiveness of automated data science workflows.

Autonomous Data Agents further extend this concept by introducing specialized agents that collaborate to analyze data and support intelligent decision-making processes [11]. By distributing responsibilities among multiple agents, the system achieves greater flexibility and adaptability when addressing complex analytical tasks.

AutoM3L represents another significant advancement in agent-based ML automation [12]. The framework utilizes multiple intelligent agents coordinated through LLMs to automate ML activities while maintaining modularity and extensibility.

Although these systems demonstrate the potential of agent-based ML automation, several limitations remain. Many focus on specific aspects of the ML workflow, provide limited deployment support, or lack comprehensive explainability mechanisms. Furthermore, fully integrated end-to-end systems that automate the entire journey from raw data to deployment remain relatively uncommon.

Table 2.1: Literature Review Summary Table

Paper / System	Main Problem Addressed	Key Methods / Framework	Techniques	Impact / Performance	D.T.D Integration
Auto-Sklearn [5]	Complex pipeline search & manual HPO overhead.	Bayesian optimization, meta-learning, & ensembles.	Dataset meta-matching, hyperparameter tuning.	Strong out-of-the-box baseline; standard benchmark.	Integrated as a downstream model training task.
TPOT [6]	Heavy manual pipeline engineering.	Genetic Programming (GP) evolutionary search.	Crossover & mutation of pipeline components.	Finds novel pipeline designs; high compute cost.	Guides model search scope; offloaded to backend.
H2O AutoML [7]	Poor scalability & usability for large datasets.	Scalable, industrial AutoML platform.	Grid/random search, stacked ensembling (Super Learners).	Highly scalable; lacks custom pipeline flexibility.	Integrates enterprise-scale engines as AutoML candidates.
AutoGluon [8]	Suboptimal tuning & ensemble selection.	Multi-layer stack ensembling.	Stacked ensembling, layer bagging, and auto-tuning.	State-of-the-art accuracy; low code footprint.	Adopted as a core candidate engine in the AutoML node.
Pandas Profiling [2]	Manual, repetitive EDA.	Automated data profiling.	Descriptive statistics, correlation & anomaly checks.	Rapid initial data overview; static reporting only.	Baseline for raw/clean analysis with added LLM insights.
Data Interpreter [14]	Rigid analysis steps; lack of adaptive reasoning.	LLM with local code interpreter.	Dynamic code generation, execution, & debugging.	Highly adaptive; prone to code errors/hallucinations.	Emulates code interpreter execution with human checkpoints.
DS-Agent [10]	LLM failure in complex tasks	CBR & Tool-use.	Code-history recall, tool	High success on Kaggle; lacks	Adapts self-debugging loops for

	without refinement.		execution, and self-debugging.	full-pipeline scope.	code-gen stages.
Autonomous Data Agents [11]	Monolithic single-agent task bottlenecks.	Multi-agent collaboration.	Specialized agent roles (planning, coding, auditing).	High flexibility; significant API & orchestration cost.	Adapts specialized agent roles in dynamic workflows.
AutoM3L [12]	Hardcoded workflows & poor agent modularity.	Modular, LLM-coordinated agents.	Distributing ML tasks across dedicated agents.	Modular and extensible; lacks deployment support.	Maps modular stages into a compiled LangGraph state graph.

2.5 Commonly Used Technologies

Modern autoML and agent-based data science systems rely on a variety of technologies that support data processing, model development, intelligent reasoning, workflow orchestration, data management, visualization, testing, and deployment. The most commonly used technologies are summarized as follows.

Data Processing and Analysis:

- **Data Analysis Libraries (Pandas, NumPy):** Used for data manipulation, transformation, statistical analysis, and numerical computation.
- **Data Profiling Tools (Pandas Profiling, Sweetviz, AutoViz):** Generate automated EDA reports and dataset summaries.
- **Distributed Processing Frameworks (Dask, Apache Spark):** Enable scalable processing of large datasets through parallel and distributed computation.

ML and AutoML Technologies:

- **ML Frameworks (Scikit-Learn, XGBoost, LightGBM, CatBoost):** Provide algorithms for classification, regression, clustering, feature selection, and model evaluation.
- **AutoML Frameworks (Auto-Sklearn, TPOT, H2O AutoML, AutoGluon):** Automate tasks such as model selection, HPO, and pipeline generation.
- **Hyperparameter Optimization Tools (Optuna, Hyperopt, Ray Tune):** Search for optimal model configurations to improve predictive performance while reducing manual effort.

Large Language Models and Agent Technologies:

- **LLMs (GPT, Gemini, Claude, LLaMA):** Support natural language understanding, reasoning, code generation, and decision-making tasks.
- **Agent Frameworks (LangGraph, LangChain, LlamaIndex):** Enable the orchestration of intelligent agents, workflow management, and tool integration.
- **MASs:** Allow specialized agents to collaborate on different stages of the ML lifecycle, improving modularity and automation capabilities.

Backend and Data Management Technologies:

- **API Frameworks (FastAPI, Flask):** Provide communication between ML services, UIs, and external applications through web-based APIs.
- **Database Systems (MongoDB, PostgreSQL, MySQL):** Store datasets, workflow states, execution metadata, model outputs, and analytical reports.
- **Database Connectivity Tools (PyMongo):** Enable interaction between applications and database systems.
- **Data Validation Frameworks (eg. Pydantic):** Ensure consistency and correctness of structured data and machine-generated outputs.

Visualization and User Interaction Technologies:

- **Frontend Frameworks (React, Vue.js, Angular):** Support the development of interactive web applications for dataset management and result visualization.
- **Visualization Libraries (Matplotlib, Seaborn, Plotly, React Flow):** Provide graphical representations of datasets, model performance, workflow execution, and analytical insights.

Deployment, Testing, and Monitoring Technologies:

- **Deployment Technologies (FastAPI, Uvicorn):** Facilitate the deployment and execution of ML services and APIs.
- **Testing Frameworks (Pytest):** Support unit and integration testing to ensure system reliability and correctness.
- **Logging and Monitoring Tools:** Enable tracking of workflow execution, system performance, and error reporting.

These technologies collectively provide the foundation for modern AutoML and intelligent data science systems. By combining autoML techniques, LLMs, intelligent agents, scalable data processing frameworks, and deployment technologies, organizations can develop end-to-end solutions that reduce manual effort while improving efficiency, scalability, and accessibility.



Chapter Three

Proposed System

Chapter 3 Proposed System

3.1 System Overview

The D.T.D system is an intelligent AutoML platform that automates the ML lifecycle using Large Language Models (LLMs), multi-agent orchestration, and AutoML technologies. The system enables users to upload datasets, specify analytical objectives through natural language, and receive ADA, preprocessing, model training, evaluation, and deployment recommendations. The platform provides two execution modes. The first is a fully autonomous static pipeline that executes predefined ML stages sequentially. The second is a dynamic agent-based pipeline that incorporates HITL interaction [3], allowing users to review, approve, or modify decisions at different stages of execution. The system is built using FastAPI, LangGraph, MongoDB, and various ML frameworks to provide a scalable and interactive ML environment.

3.1.1 System Architecture

Figure 3.1 below illustrates the overall architecture of the D.T.D system. The architecture consists of four main layers: Input Layer, API and Persistence Layer, LangGraph Orchestration Layer, and Output Layer.

The **Input Layer** serves as the entry point of the system, where users provide datasets and task requirements through the web interface.

The **API and Persistence Layer** manage communication between the frontend and backend. It is responsible for handling requests, managing pipeline execution, storing workflow states and metrics in MongoDB, maintaining cached results, and recovering interrupted executions.

The **LangGraph Orchestration Layer** [1] forms the core of the system and supports two execution modes. The first is a static pipeline that executes a predefined sequence of stages including data analysis, preprocessing, and AutoML training. The second is a dynamic multi-agent pipeline that employs specialized agents for intent detection, EDA, preprocessing, feature engineering, model selection, training, and deployment. HITL

checkpoints are integrated throughout the dynamic pipeline, allowing users to review results and provide feedback before execution proceeds.

The orchestration layer interacts with several ML and AutoML technologies, like Scikit-Learn, AutoGluon, XGBoost, Optuna, and Dask, which perform the underlying tasks.

Finally, the **Output Layer** provides users with generated reports, visualizations, processed datasets, evaluation metrics, and trained ML models. This layered architecture ensures support for both fully automated and interactive ML workflows.

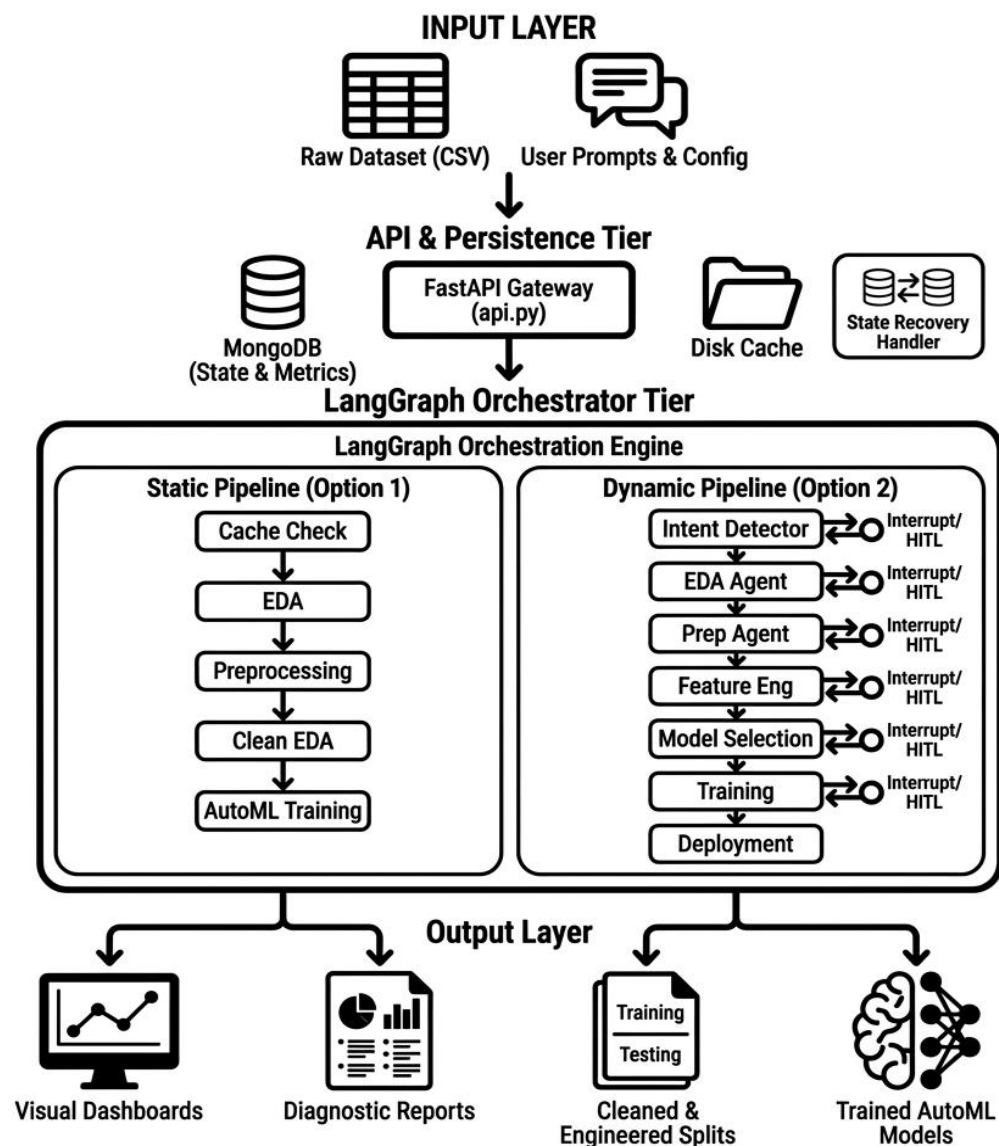


Figure 3.1: D.T.D System Architecture

3.1.2 Functional Requirements

The D.T.D system shall provide the following functionalities:

1. Upload datasets in comma-separated values (CSV) format.
2. Accept natural language user requests.
3. Automatically identify dataset characteristics and target variables.
4. Perform EDA.
5. Generate automated data quality reports.
6. Execute data preprocessing and cleaning operations.
7. Perform feature engineering.
8. Select suitable ML models automatically.
9. Train and evaluate ML models.
10. Generate model performance reports and visualizations.
11. Allow users to review and modify agent decisions through HITL checkpoints.
12. Resume interrupted pipeline executions.
13. Store execution history and pipeline states.
14. Generate deployment-ready ML artifacts.
15. Provide real-time progress tracking during execution.

3.1.3 Nonfunctional Requirements

The system should support processing large datasets through distributed computing technologies such as Dask and scalable ML frameworks.

1. Reliability

Pipeline states should be persistently stored to allow recovery from failures and unexpected interruptions.

2. Usability

Users should be able to operate the system through an intuitive web interface without requiring advanced ML expertise.

3. Performance

The system should minimize execution time through caching mechanisms, parallel processing, and optimized ML workflows.

4. Maintainability

The multi-agent architecture should support modular development and independent enhancement of individual agents.

5. Extensibility

New agents, tools, and ML algorithms should be easily integrated into the existing architecture.

6. Explainability

The system should provide detailed reports, visualizations, and intermediate outputs to help users understand pipeline decisions.

7. Availability

The platform should support continuous operation and recovery of interrupted executions through state persistence mechanisms.

3.1.4 System Users

A. Intended Users

Data Scientists

Data scientists use the platform to automate repetitive ML tasks, accelerate experimentation, and evaluate multiple models efficiently.

ML Engineers

ML engineers use the system to build, evaluate, and deploy ML pipelines with reduced manual effort.

Researchers and Students

Researchers and students can utilize the platform to perform data analysis and ML experiments without extensive implementation overhead.

Business Analysts and Domain Experts

Users with limited ML expertise can leverage the natural language interface and HITL capabilities to obtain predictive insights from data.

B. User Characteristics

The system is designed for users with varying levels of technical expertise.

Basic Users

- Familiarity with datasets and spreadsheets.
- Ability to upload data and interpret generated reports.
- No advanced programming knowledge required.

Intermediate Users

- Understanding of ML concepts.
- Ability to review and provide feedback to intelligent agents.
- Capability to interpret model evaluation metrics.

Advanced Users

- Experience with data science and ML workflows.
- Ability to customize analytical objectives and evaluate system recommendations.
- Knowledge of model deployment and performance optimization techniques.

This broad user spectrum aligns with the primary goal of D.T.D, which is to make ML more accessible while still providing advanced functionality for experienced practitioners.

3.2 System Analysis & Design

3.2.1 Use Case Diagram

The functional requirements, user-system boundaries, and actor interactions of the D.T.D AutoML platform are mapped out in the Use Case Diagram shown in Figure 3.2. The diagram isolates the system boundary, the human roles, and the external dependencies supporting backend processes.

1. System Actors

The platform coordinates actions between two human actors and two secondary supporting systems:

User / Data Analyst / AI/ML Engineer (Primary Human Actor): Represents the end-user interacting directly with the frontend web interface. This actor initializes sessions, configures workflows, monitors progress, and steers execution through checkpoint decisions.

System Admin (Human Actor): Represents the administrator who manages user accounts, reviews system complaints or messages, monitors system performance statistics, and exports compiled model splits and plots.

LLM API Provider (Secondary System Actor): An external service (e.g., Gemini API) invoked to process unstructured natural language queries and extract structured schemas during intent detection, as well as to power cognitive prompt-adjustments across the multi-agent dynamic pipeline.

MongoDB Database Store (Secondary System Actor): The persistent storage layer that handles data lookups, records session metrics, stores user accounts, manages complaints/messages, and updates agent checkpoints. It is utilized by almost all primary use cases to maintain system persistence.

2. Functional Use Cases and Relationships

The diagram defines **twelve** core use cases inside the system boundary, structured across the system's operational lifecycle and administrative functions:

Session Ingestion & Configuration: The user initiates a run via Initialize Session & Upload Dataset and Configure & Select Execution Option. If the dataset schema is complex or the target is unspecified, they can invoke Query Target Suggestion to receive automated column recommendations from the backend.

Autonomous Pipeline Execution: The Execute Autonomous Static Pipeline use case has an <<include>> relationship with Cache Check. This indicates that checking for pre-existing dataset hashes is a mandatory step that executes automatically during every static run to skip redundant operations.

Interactive Pipeline Execution: The Execute Interactive Dynamic Pipeline use case is dynamically customized through an <<extend>> relationship with LLM Intent Detection. The intent detection module is triggered conditionally to parse user natural language queries and schema types via the LLM API Provider and MongoDB Database Store to establish routing flags. In standard UML notation, the <<extend>> arrow points from the extension use case (LLM Intent Detection) back to the base use case (Execute Interactive Dynamic Pipeline).

HITL Controls: The user tracks step-by-step agent actions via Monitor Progress checklist (via sub_nodes). When the orchestrator hits a checkpoint node, the user interacts via Submit Feedback & Resume (HITL checkpoint) to send decisions and natural language adjustments back to the FastAPI engine.

Fault Tolerance & Output: If a connection drops or the server restarts, the Submit Feedback & Resume flow automatically triggers Trigger In-Memory Thread Recovery (modeled as an <<include>> relationship) to restore the in-memory LangGraph checkpoint from MongoDB. Upon pipeline completion, both the User and System Admin can invoke Export Trained Models, Splits, & Plots to download compiled artifacts.

Administrative Controls (Admin-Only): Restricted to the System Admin actor, these use cases manage platform-level operations:

- **Manage User Accounts:** Allows the admin to view, create, and manage user accounts and roles.
- **Review Complaints & Messages:** Enables the admin to review user-submitted reports or complaints and update contact statuses.
- **Monitor System Statistics:** Aggregates database sizes, user counts, total reports, and total pipeline runtimes to monitor overall platform health.



Figure 3.2: D.T.D Use Case Diagram

3.2.2 Class Diagram

Class Descriptions

1. Core Orchestration Classes

As shown in figure 3.3:

DTDPipeline: The primary entry-point class for the static execution pipeline. It initializes the workspace environment, holds the configuration parameters (run_id, state), and manages static runs by checking the cache before executing sequential data engineering tasks.

ControllerAgent: The main orchestrator class that builds and manages the LangGraph StateGraph for interactive, dynamic runs. It registers the checkpoint in-memory store (MemorySaver), handles interrupts, and manages session recovery loops. It coordinates the routing of the dataset state through all subsequent agent modules.

2. State & Configuration Schemas

PipelineState (TypedDict): The global shared state schema running through the execution graph. It contains metadata fields such as input file paths, target column definitions, identified task types (classification/regression), intent flags, intermediate agent output lists, and human feedback history records.

IntentFlags: A contract mapping boolean routing flags (eda, preprocessing, feature_eng, model_selection, training, deployment). The intent detector agent populates this object via structured LLM calls to map out the active pipeline execution nodes.

3. Agent Execution Classes

IntentDetectorAgent: Responsible for natural language instruction parsing. Its primary method, detect_intent(), uses structured LLM prompting to map the user's instructions to the IntentFlags object.

EDAAgent: The EDA profiler. Its methods (generate_insights()) and (compute_target_analysis()) compute dataset statistics, missing values metrics, and Spearman/Pearson correlation coefficients to output target column suggestions.

PreprocessingAgent: Inherits core properties from the controller agent and handles dataset cleaning operations. Its methods (`clean_dataset()`, `balance_imbalance()`, `split_train_test()`) apply mathematical transformations, including Z-score scaling, one-hot encoding, and SMOTE over-sampling.

FeatureEngineeringAgent: Responsible for continuous and text feature creation. Its methods (`expand_features()`, `engineer_interactions()`) generate candidate features and filter them using correlation scoring metrics against the target column.

AutoMLAgent: Manages model tuning and ensembling. Its primary methods handle the Bayesian HPO search loop via Optuna (`tune_hyperparameters()`), stack ensembling via AutoGluon (`train_models()`) and compiling diagnostic metrics and plots (`_run_evaluation_subnode()`).

4. Utility and Storage Classes

PipelineCacheManager: Handles cached states. Its methods (`check_cache()`, `save_state_to_cache()`) hash dataset binaries and target variables to retrieve cached outputs, skipping execution stages for identical runs.

DatasetLoader: A helper class that processes raw tabular files. Its methods (`load_csv()`, `extract_schema()`) load chunks of data and map physical column types to system datatypes.

MongoDatabaseConnector: The primary database connection class. Its methods (`store_initial_knowledge_graph()`, `update_agent_progress()`) update the user checkpoint states in MongoDB collections, enabling real-time status updates on the client UI.

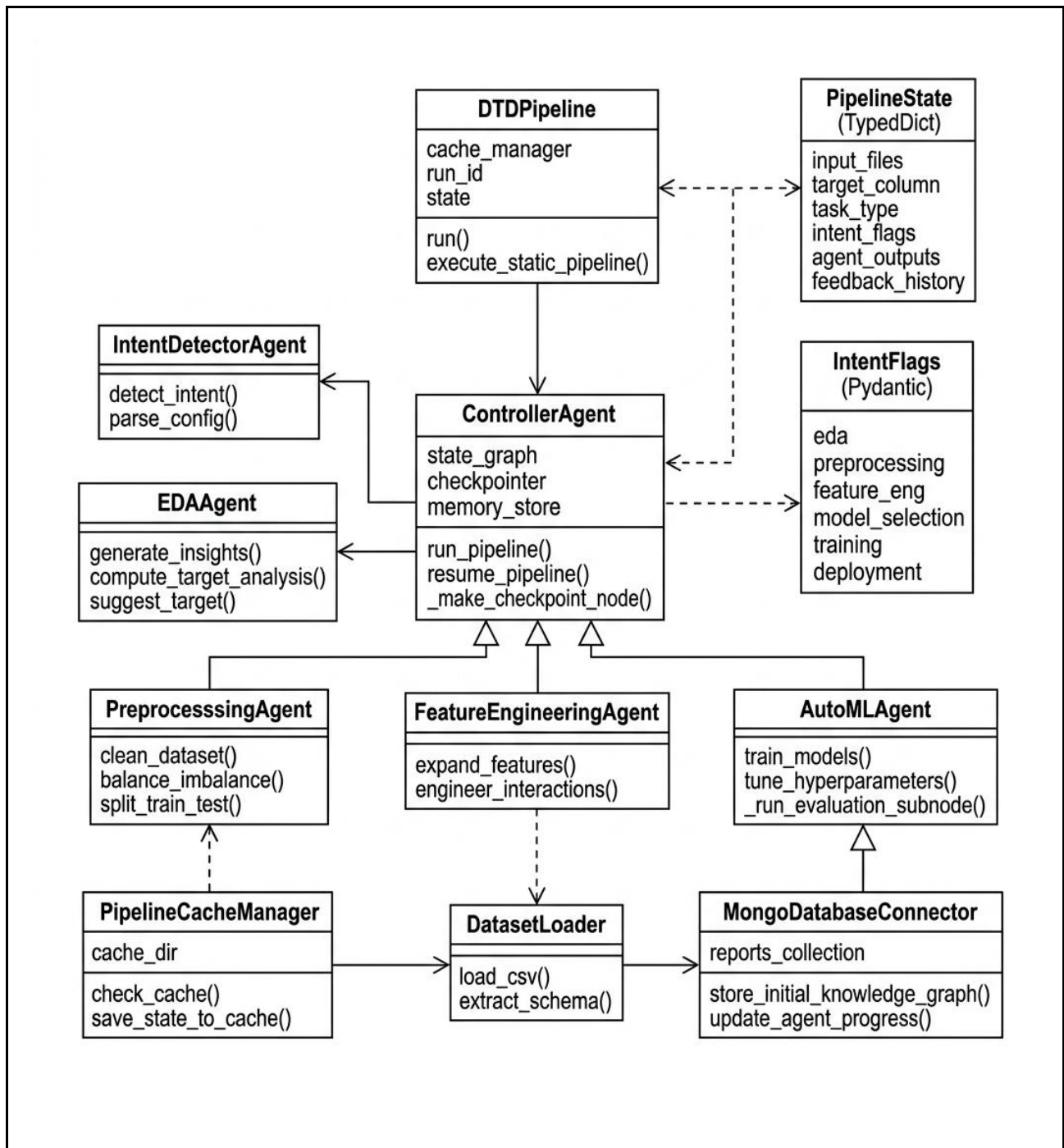


Figure 3.3: D.T.D Class Diagram

3.2.3 Sequence Diagrams

Static Pipeline Execution with Caching Optimization

The diagram below demonstrates the end-to-end flow of the fully automated Static Pipeline. It illustrates how the system optimizes processing time by utilizing a content-aware caching layer before resorting to computationally heavy agent nodes (exploratory analysis, preprocessing, and training).

Core Workflow Description

1. **Trigger & Cache Lookup:** The user uploads a dataset and chooses "Static Mode". The frontend sends a POST `/run-pipeline/{dataset_id}` request to the backend. The backend hashes the raw dataset file bytes and target column name, querying the Cache Manager.
2. **Alternative Control Flows:**
 - **Cache Hit:** If matching cached records exist, the Cache Manager returns the stored outputs. The backend immediately streams these pre-computed results to the frontend via Server-Sent Events (SSE).
 - **Cache Miss:** If no cache matches, the backend initiates the Static StateGraph execution:
 - **Stage 1 (Profile):** Runs `raw_analysis` to profile the dataset.
 - **Stage 2 (Preprocess):** Runs preprocessing to impute missing values, scale, and split the data.
 - **Stage 3 (Re-Profile):** Runs `clean_analysis` on the preprocessed dataset.
 - **Stage 4 (Model):** Invokes `automl_training` using engines like AutoGluon or Scikit-Learn.
3. **Post-Execution Operations:** Once the model is trained, the metrics and serialized model artifact are returned to the backend. The backend writes the fresh outputs to the Cache disk, logs the performance metrics in MongoDB, and streams progress updates using SSE.

4. Presentation: The frontend processes the streams and displays the completed model leaderboard to the user.

Key Technical Insights

- Deterministic Content Caching: Caching is driven by data hashing (bytes + prediction target), ensuring that identical datasets and targets bypass repetitive preprocessing and training loops.
- Server-Sent Events (SSE): Rather than relying on simple REST polling, the system uses asynchronous SSE streaming to keep the client UI continuously updated with granular agent status checklist logs.

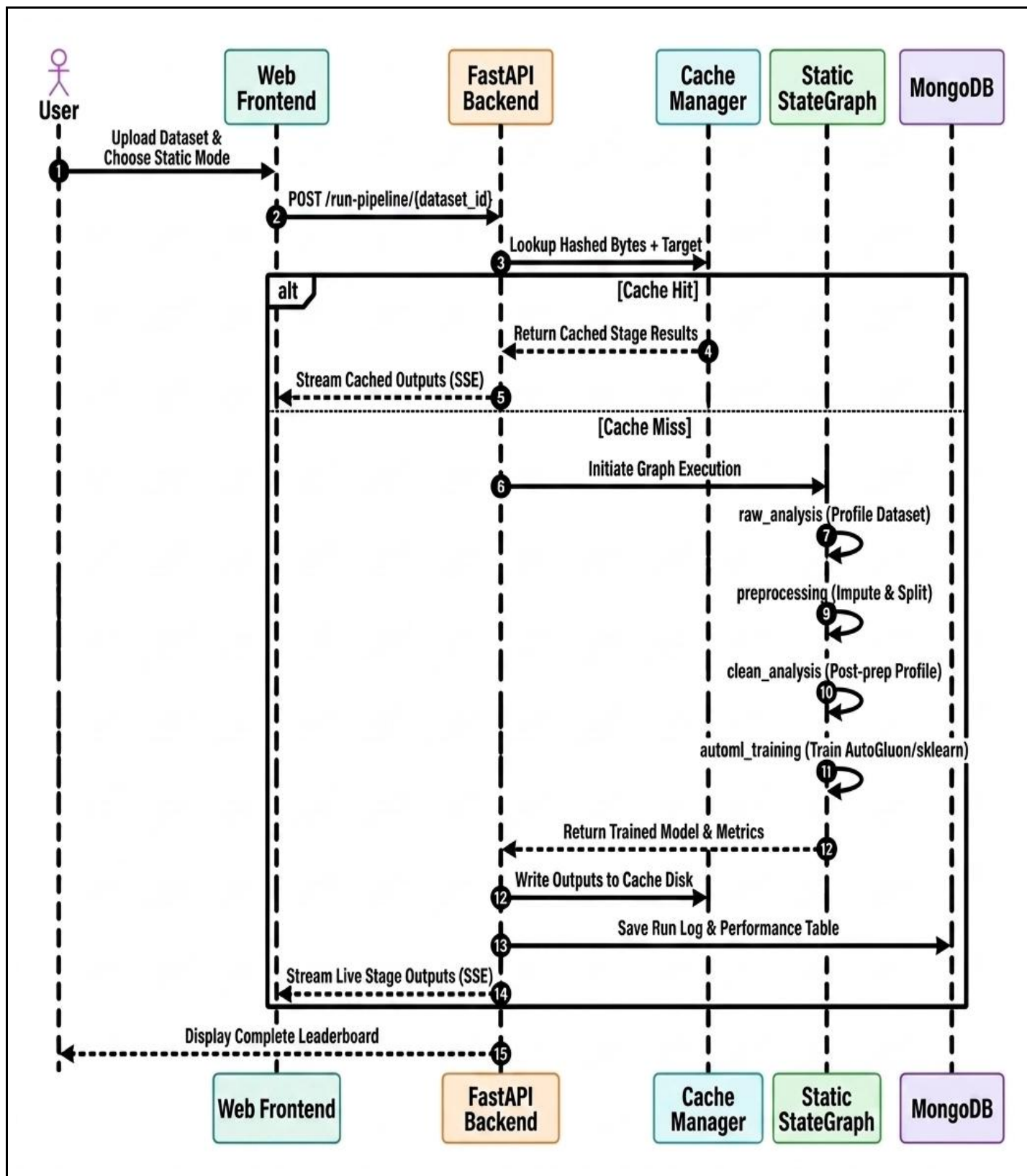


Figure 3.4: D.T.D Static Pipeline Sequence Diagram

Dynamic Pipeline Execution with HITL Interactivity

The diagram below illustrates the control flow and communication protocols of the Dynamic Pipeline option. It highlights the platform's HITL capabilities, showing how the orchestrator integrates user guidance, reviews, and manual overrides during a multi-agent ML workflow (specifically shifting between EDA and preprocessing stages).

Core Workflow Description

1. **Pipeline Initialization:** The user initiates a run from the Web Frontend, which sends a POST `/dynamic/start` request to the FastAPI Backend.
2. **StateGraph Execution & Interrupt:** The backend compiles and executes the Dynamic StateGraph (managed via LangGraph). The graph runs until the `eda_agent` completes dataset profiling, triggers a checkpoint `interrupt()`, and yields a paused state with a unique thread ID.
3. **State Persistence:** The backend catches the paused state, stores a serialized state snapshot in MongoDB (setting the status to "paused"), and returns a JavaScript object notation (JSON) payload to the frontend.
4. **Interactive Audit Loop:** The frontend displays the proposed plan to the user, who reviews the output and returns custom text feedback or edits via a POST `/dynamic/resume/run_id` request containing the feedback payload.
5. **State Recovery & Resumption:** On receive, the backend fetches the corresponding state document from MongoDB to restore the checkpoint, re-seeding the execution pointer. It then resumes the state graph, applying the user's feedback.
6. **Next Agent Stage:** The graph triggers the `preprocessing_agent` to apply the modified directives, which runs until it hits the subsequent checkpoint `interrupt`, restarting the user validation cycle.

Key Technical Insights

- **State Recovery Resiliency:** Demonstrates how state snapshots are persisted to MongoDB. If the FastAPI application server undergoes a restart, the checkpoint state is recovered from database records, preventing execution progress loss.
- **Granular Checkpointing:** Illustrates the use of execution checkpoints (`interrupt()`) to halt graph progression and capture structured validation points for human review.

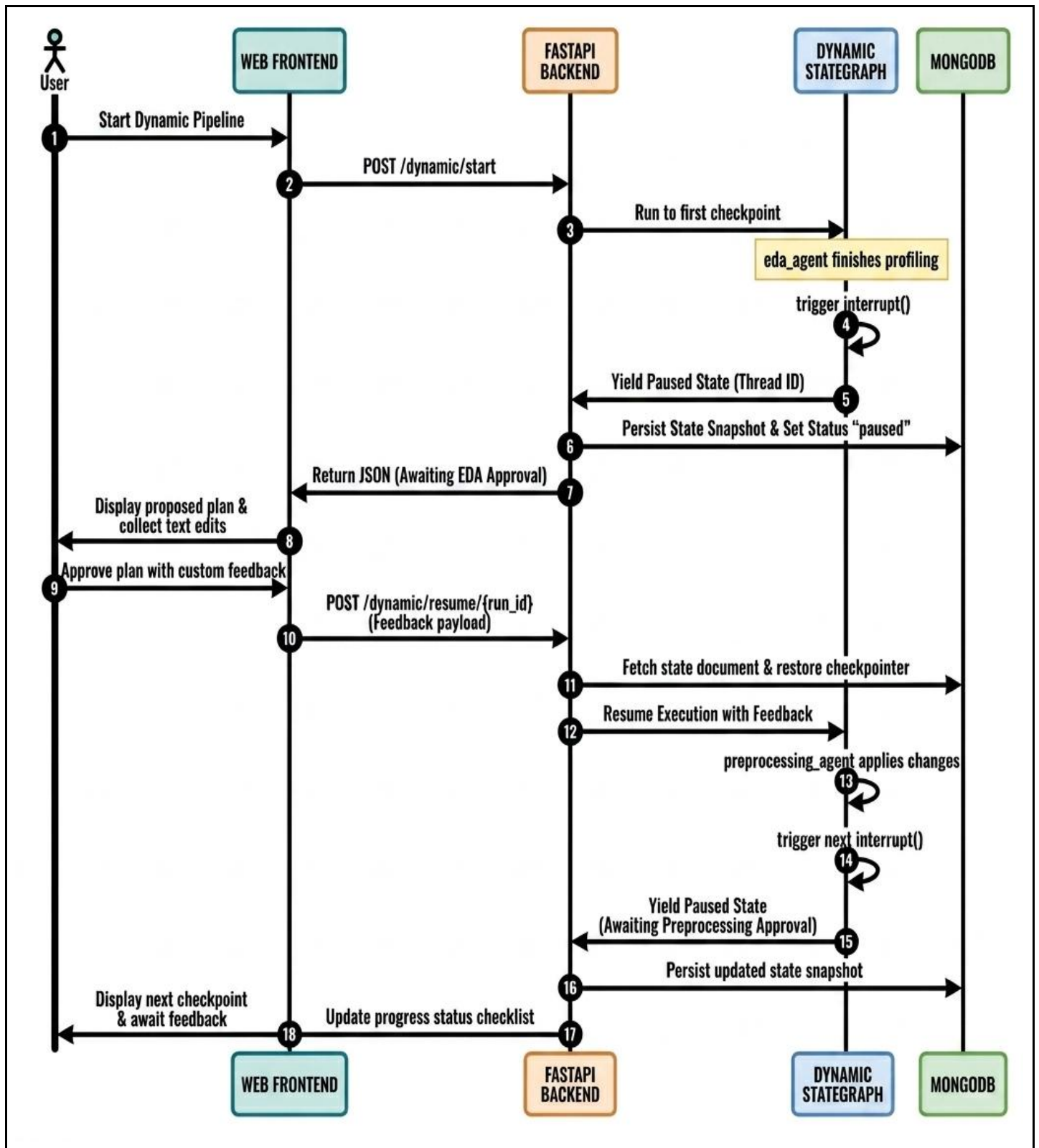


Figure 3.5: D.T.D Dynamic Pipeline Sequence Diagram

3.2.4 Database Diagram

The D.T.D database adopts a document-oriented relational model using MongoDB, managed through the Mongoose ODM. Although MongoDB is schema-less by nature, Mongoose enforces structured schemas at the application layer, providing type validation, required field constraints, and referential integrity through ObjectId foreign keys.

The reports collection occupies the central position in the schema topology. It acts as a junction between users and datasets, and is referenced outwardly by complaints. Compound indexes on { `userId`, `createdAt` } and { `userId`, `isStarred`, `createdAt` } are declared to optimize the most frequent dashboard query patterns — retrieving a user's full report history and their starred reports, both sorted in reverse chronological order.

The contacts collection is intentionally decoupled from all other entities, operating as a standalone support channel with no foreign key relationships, reflecting its independence from the data pipeline workflow. Below is figure 3.6 and table 3.1 to further understand the database collections.

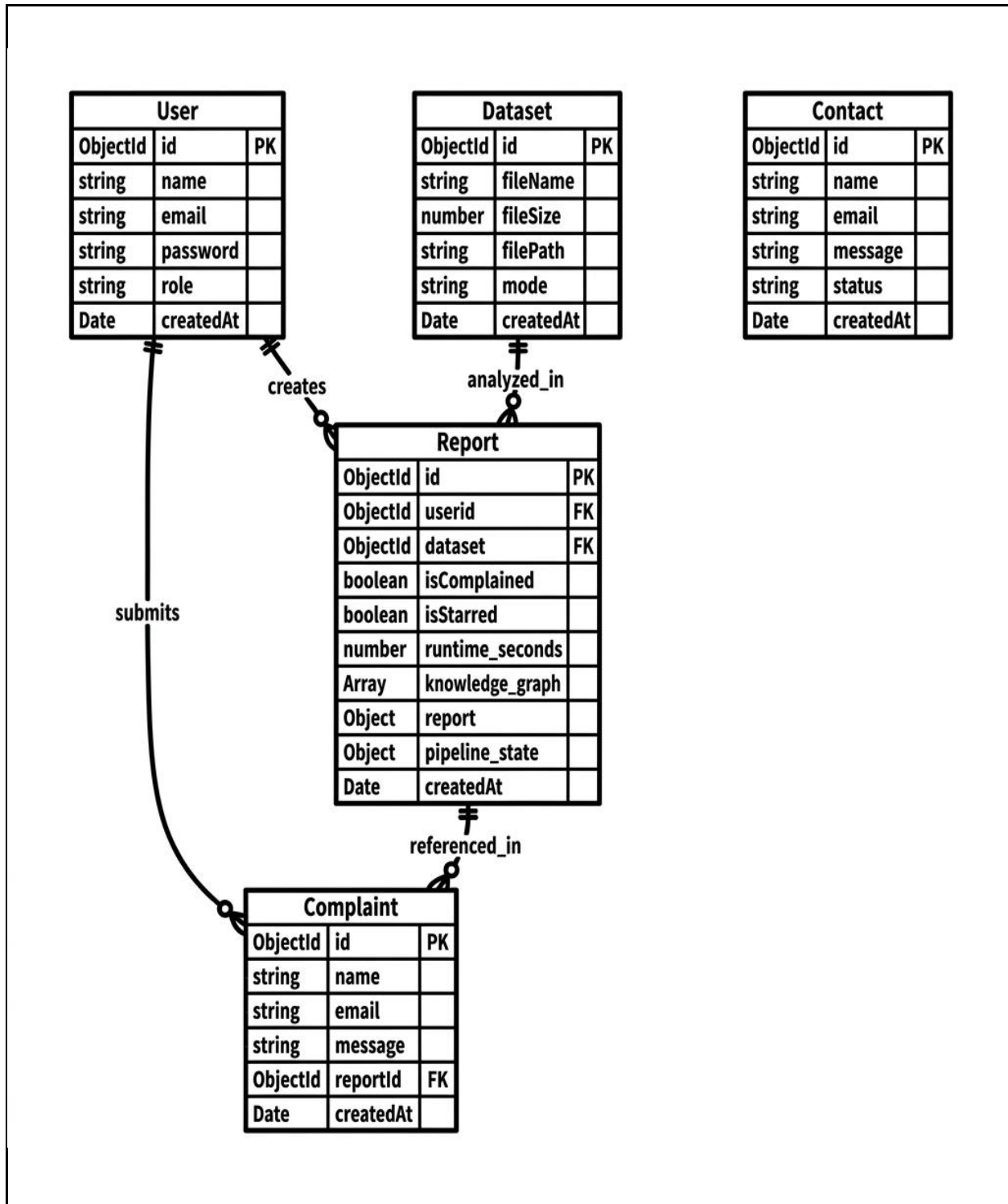


Figure 3.6: MongoDB ERD Database Schema Diagram

Table 3.1: MongoDB Collections and their roles in the D.T.D System

Collection	Purpose
users	Stores user account credentials, profile information, access roles (<i>USER / ADMIN</i>), and password recovery tokens. Serves as the authentication and authorization anchor for all user-owned resources in the system.
datasets	Holds metadata for every dataset uploaded by a user, including the original file name, size in bytes, storage path, and the processing configuration (prompt and mode) used to initiate the pipeline. Acts as the entry point that links a file to its downstream analysis.
reports	Central collection representing a complete pipeline execution record. Stores references to the owning user and source dataset, execution timing metrics (<i>start_time</i> , <i>end_time</i> , <i>runtime_seconds</i>), the full analytical output payload, and extracted KG entities. Serves as the primary read target for the frontend dashboard.
complaints	Captures structured feedback submitted by users when they identify inaccuracies or quality issues in a specific report. Each complaint is linked to its target report via a foreign key reference, enabling administrators to triage and audit model output quality.
contacts	Logs general-purpose inquiries and messages submitted through the platform's public contact form. Each entry carries a lifecycle status (<i>RECEIVED</i> , <i>PENDING</i> , <i>REPLIED</i>) to support administrative triage and response tracking, independently of the core pipeline workflow.

Entity Relationship Summary

Table 3.2 outlines the core one-to-many (1:N) and one-to-one (1:1) relationships within the D.T.D. system. Each "Parent" entity may or may not be associated with multiple records in the corresponding "Child" entity.

Table 3.2: Core One-to-Many relationships in the D.T.D System

Parent	Child	Relationship	Description
users	reports	One-to-Many (1:N)	A registered user can own and initiate multiple pipeline execution reports. Each report document stores a <i>userId</i> foreign key referencing the owning user, ensuring full traceability of analytical outputs to their originating account.
datasets	reports	One-to-One (1:1)	Each uploaded dataset corresponds to a single pipeline execution and generates a single report. The report maintains a reference to its source dataset, enabling traceability between the generated results and the uploaded data artifact.
reports	complaints	One-to-Many (1:N)	A report may accumulate multiple user-submitted complaints if recurring inaccuracies or quality issues are identified across different sessions. Each complaint carries a <i>reportId</i> foreign key that anchors it to the specific report under review, supporting structured quality auditing.



Chapter Four

System Implementation and Testing

Chapter 4 System Implementation and Testing

4.1 Datasets

The D.T.D AutoML platform is designed to ingest, analyze, and train ML pipelines on structured tabular datasets. To evaluate and verify the system's multi-agent capabilities, the platform is integrated with three classic benchmark datasets representing distinct ML tasks, dimensionalities, distributions, and preprocessing challenges. Additionally, the system supports generic user-uploaded files, validating raw schemas dynamically during execution.

4.1.1 Titanic Dataset (Binary Classification)

The Titanic passengers dataset is utilized as the primary benchmark for binary classification. The objective is to predict passenger survival based on demographic and trip-related characteristics.

- **Dimensionality:** 891 instances (rows) by 11 features (columns).
- **Target Variable:** Survived (Binary integer: *0* representing non-survival, *1* representing survival).
- **Target Distribution:** 342 survived (38.38%) and 549 did not survive (61.62%). This distribution represents a mild class imbalance, requiring target stratification during dataset splitting and minor class weight adjustments.
- **Feature Schema:**
 - *Numerical Continuous:* Age (numeric passenger age), Fare (ticket transaction cost).
 - *Numerical Discrete (Integer):* SibSp (count of siblings and spouses aboard), Parch (count of parents and children aboard), Pclass (socio-economic status ticket class: 1st, 2nd, or 3rd).
 - *Categorical Nominal:* Sex (gender identifier), Embarked (port of embarkation: C = Cherbourg, Q = Queenstown, S = Southampton).

- *High-Cardinality Text*: Name (passenger names), Ticket (alphanumeric ticket identifiers), Cabin (cabin room locations).
- **Implementation Challenges**:
 - *Missing Values*: The feature Age contains 177 missing values (19.8%), requiring median-based numerical imputation. The feature Cabin contains 687 missing records (77.1%), requiring constant-label categorical imputation or drop strategies. The feature Embarked contains 2 missing entries, imputed using the statistical mode.
 - *High Cardinality*: Alphanumeric features like Ticket and Cabin require target encoding or structural parsing to prevent feature explosion under standard one-hot encoders.

4.1.2 Iris Dataset (Multi-Class Classification)

The classic Iris botanical dataset serves as a low-complexity benchmark to verify multi-class classification execution and assess the system's baseline model selection behaviors.

- **Dimensionality**: 150 instances by 4 features.
- **Target Variable**: target (Nominal categorical, mapped to three distinct flower species: *Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*).
- **Target Distribution**: Exactly 50 instances per class (33.3%), representing a perfectly balanced class distribution.
- **Feature Schema**:
 - *Numerical Continuous*: sepal length (cm), sepal width (cm), petal length (cm), petal width (cm).
- **Implementation Challenges**:
 - *Zero Missing Values*: There are no missing coordinates in the Iris dataset, allowing testing of bypass routing inside the preprocessing agent.
 - *Low Dimensionality*: Because features are numeric and highly separable, this dataset acts as a control test to evaluate if the system's AutoML tuning

selects simpler linear estimators (e.g., Logistic Regression) over complex stacked ensembles to prevent overfitting.

4.1.3 Wine Quality Dataset (Multi-Class Classification)

The Wine chemical dataset is used to benchmark multi-class prediction on continuous features with wide variations in scales and distributions, representing a typical high-dimensional chemical analysis task.

- **Dimensionality:** 178 instances by 13 features.
- **Target Variable:** target (Nominal categorical, representing three distinct wine cultivars grown in the same region in Italy).
- **Target Distribution:** Cultivar 1 (59 samples), Cultivar 2 (71 samples), and Cultivar 3 (48 samples).
- **Feature Schema:**
 - *Numerical Continuous:* alcohol, malic_acid, ash, alcalinity_of_ash, magnesium, total_phenols, flavanoids, nonflavanoid_phenols, proanthocyanidins, color_intensity, hue, od280/od315_of_diluted_wines, and proline.
- **Implementation Challenges:**
 - *Feature Scale Variation:* Proline values range from 278 to 1680, whereas nonflavanoid phenols range from 0.13 to 0.66. This variance requires standardized scaling (Z-score) or robust scaling to ensure distance-based classifiers are not dominated by high-magnitude variables.

4.1.4 Generic Dataset Upload (CSV Ingestion)

The D.T.D platform supports arbitrary user-uploaded CSV tabular files. When a user uploads a new dataset, the system performs an asynchronous schema verification check. This check detects the count of rows and columns, maps datatypes to physical storage objects, identifies target candidate columns, and flags potential null coordinates.

The validation engine has been verified to handle large-scale datasets, such as the Airbnb_Master.csv (~11.8MB) and car_prices.csv (~88MB) uploaded files. The system processes these large files by chunking data streams and building lazy-loaded indexes in MongoDB to prevent memory overflows during execution.

Table 4.1: Dataset Characteristics Summary

Dataset Name	ML Task	Dimensions (Rows x Features)	Target Variable	Class Distribution	Missing Values Count
Titanic	Binary Classification	<i>891 times 11</i>	Survived	38.38% Survived, 61.62% Died	Age (177), Cabin (687), Embarked (2)
Iris	Multi-class Classification	<i>150 times 4</i>	target	Balanced (3 classes of 50 samples each)	None (0)
Wine Quality	Multi-class Classification	<i>178 times 13</i>	target	Cultivar 1 (59), 2 (71), 3 (48)	None (0)

4.2 Modules

The D.T.D platform coordinates operations through a multi-agent execution pipeline. Each task in the ML workflow is managed by a dedicated agent module containing distinct tools and execution rules. The modules communicate state changes through a centralized graph memory.

4.2.1 Intent Detection

The Intent Detection module is the initial gateway of the D.T.D agent pipeline. It translates the user's natural language queries and dataset characteristics into a structured execution configuration.

- **Component Responsibility:** Ingests raw textual instructions (e.g., "build a classifier to predict passenger survival, do not drop age") and matches the target column and model constraints against the parsed schema properties.
- **Ingestion and Schema Processing:** The module reads dataset files, extracting details like file name, size, shape, column labels, data types, and null indicators.
- **Structured Output Model:** The natural language input and schema are passed to the Gemini model inside a structured execution block. This block binds the LLM's response format to a strict contract named IntentFlags. The contract maps properties into boolean values:
 - `eda`: Determines whether to trigger exploratory profiling.
 - `preprocessing`: Determines whether data transformations are required.
 - `feature_engineering`: Specifies whether feature engineering tools should execute.
 - `model_selection`: Triggers AutoML hyperparameter optimization.
 - `training`: Triggers final model execution.
 - `evaluation`: Specifies whether diagnostic reports, ROC curves, and confusion matrices should be generated.

- `target_column`: Captures the explicit column name to predict, or remains empty to trigger target suggestion.
- `task_type`: Formats the task type to "classification", "regression", "clustering", or "unknown".
- **Routing Decisions**: Once the IntentFlags are populated, the intent detector registers the configurations into the PipelineState and routes execution to the next active agent (usually the EDA Profiler).

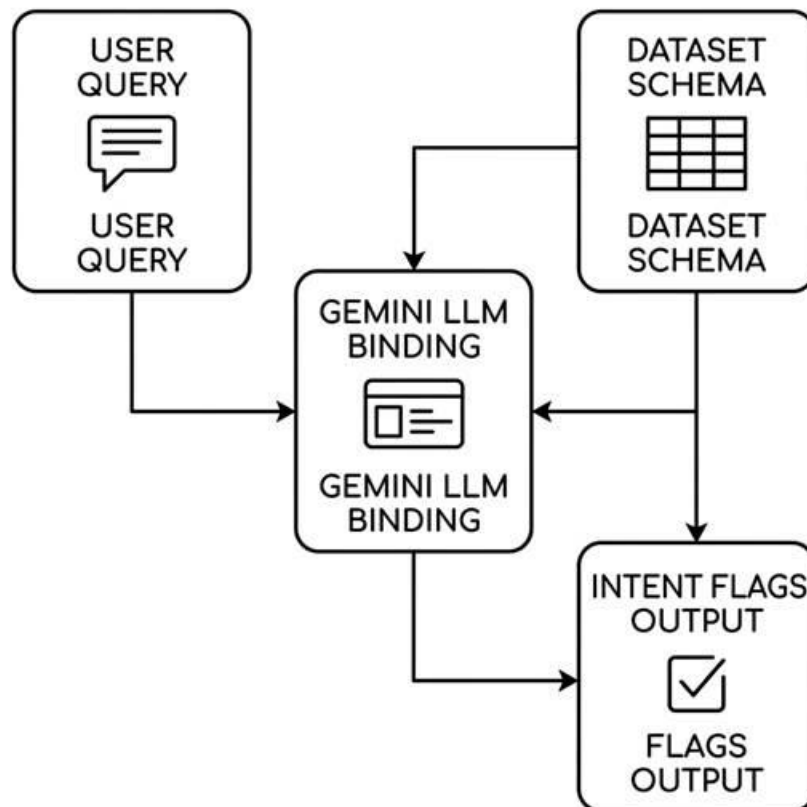


Figure 4.1: Intent Detection Processing Diagram

4.2.2 EDA (Exploratory Data Analysis)

The EDA module inspects the dataset's statistical properties. It generates baseline data profiles and resolves missing target targets before training.

- **Component Responsibility:** Evaluates dataset characteristics, column dependencies, class skews, and missing values.
- **Descriptive Statistical Calculations:** The EDA tool computes key metrics for each numeric column:

- *Mean:*

Equation 4.1

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

- *Variance:*

Equation 4.2

$$\sigma^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2$$

- *Skewness:*

Equation 4.3

$$g_1 = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^3}{\sigma^3}$$

- **Column Relationship Checks:** Pearson product-moment correlation coefficients are computed for continuous variables, while Spearman rank coefficients are calculated for ordinal columns.
- **Target Feature Suggestion:** If the target column is undefined, the agent scans columns for unique ratios and label distributions, suggesting target options to the user.
- **Output Compilation:** Generates a statistical summary, flags features with high null ratios, and appends the initial workflow layout metadata to the execution state.

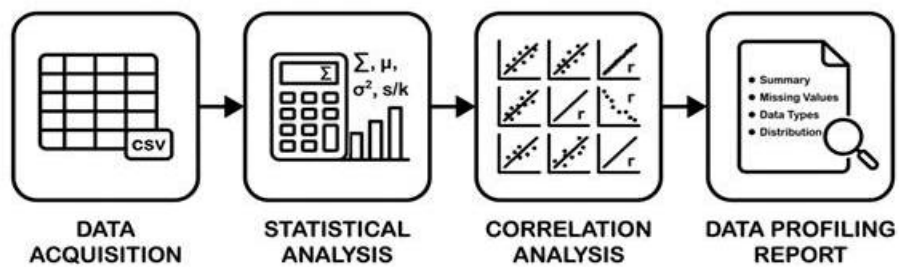


Figure 4.2: EDA Profiling and Statistical Summary Diagram

4.2.3 Preprocessing

The Preprocessing module executes data cleaning operations in a sequential pipeline to prepare training and validation splits.

- Component Responsibility:** Formulates transformation plans, splits data, imputes missing values, treats outliers, encodes categorical data, scales numerical features, and balances targets.

Table 4.2 below presents the preprocessing operations automatically selected for each feature in the Titanic dataset, including missing value handling, encoding techniques, scaling methods, and outlier treatment strategies.

Table 4.2: Feature-Specific Preprocessing Mapping (Titanic Case Study)

Feature Name	Data Type	Missing Ratio	Imputation Strategy	Encoding Applied	Scaling Applied	Outlier Handling
PassengerId	Integer (ID)	0.0%	None (Dropped)	None (Dropped)	None	None
Survived	Binary (Target)	0.0%	None	None	None	None
Pclass	Ordinal	0.0%	None	Ordinal Encoder	None	None
Name	String	0.0%	None	Target Encoder	None	None
Sex	Categorical	0.0%	None	One-Hot Encoding	None	None
Age	Continuous	19.8%	Median Imputation	None	Standard Scaling	IQR clipping
SibSp	Discrete	0.0%	None	None	None	Z-score thresholding
Parch	Discrete	0.0%	None	None	None	Z-score thresholding
Ticket	String	0.0%	None	Target Encoder	None	None
Fare	Continuous	0.0%	None	None	Robust Scaling	IQR clipping
Cabin	String	77.1%	Mode (Constant "M")	One-Hot Encoding	None	None
Embarked	Categorical	0.2%	Mode Imputation	One-Hot Encoding	None	None

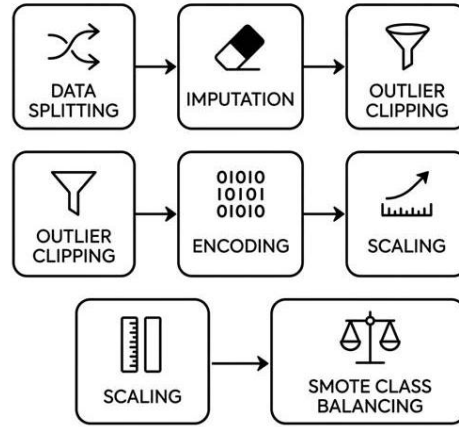


Figure 4.3: Preprocessing Sequential Transformation Diagram

4.2.3.1 Mathematical Formulations of Preprocessing transformations

Data Imputation

- **Numerical Imputation (Median):** Replaces null values with the median of the continuous column: *Equation 4.4*

$$\tilde{x} = \text{median}(x)$$

- **Categorical Imputation (Mode):** Replaces missing nominal labels with the most frequent category: *Equation 4.5*

$$\tilde{c} = \arg \max_{c_j} \sum_{i=1}^n \mathbb{I}(c_i = c_j)$$

Categorical Encoding

- **One-Hot Encoding:** Maps low-cardinality nominal variables into K binary attributes.
- **Target Encoding:** Replaces category c_j with the smoothed target expectation: *Equation 4.6*

$$S(c_j) = \frac{\sum_{i: C_i=c_j} y_i + m \cdot \bar{y}}{N(c_j) + m}$$

where $\bar{y}$ is the global target mean, $N(c_j)$ is the sample count for category c_j , and m is a smoothing parameter.

Scaling and Normalization

- **Standard Scaling (Z-score):** Scales continuous columns to zero mean and unit variance: *Equation 4.7*

$$z = \frac{x - \mu}{\sigma}$$

- **Min-Max Scaling:** Restricts variables to the interval $[0, 1]$: *Equation 4.8*

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- **Robust Scaling:** Normalizes variables using quartile bounds to resist outlier distortion: *Equation 4.9*

$$x_{\text{robust}} = \frac{x - Q_2}{Q_3 - Q_1}$$

where Q_2 is the median, and $Q_3 - Q_1$ represents the Interquartile Range (IQR).

Outlier Treatment

- **IQR Boundary Clipping:** Clips features to quartile thresholds: *Equation 4.10*

$$x_{\text{clipped}} = \max(\text{Lower Limit}, \min(x, \text{Upper Limit}))$$

$$\text{Lower Limit} = Q_1 - 1.5 \cdot IQR$$

$$\text{Upper Limit} = Q_3 + 1.5 \cdot IQR$$

- **Z-score Thresholding:** Identifies outliers exceeding standard deviation limits:

Equation 4.11

$$\text{Outliers} = \{x_i \mid |z_i| > 3\}$$

Dataset Balancing

- **Synthetic Minority Over-sampling Technique (SMOTE):** Creates synthetic minority samples x_{new} along lines connecting minority samples x_i to their nearest neighbors x_{zi} : *Equation 4.12*

$$x_{\text{new}} = x_i + \lambda(x_{zi} - x_i) \quad \text{where } \lambda \sim U(0, 1)$$

- **Random Under-Sampling (RUS):** Randomly drops samples from the majority class to balance class frequencies.
- **Weighted Loss Adjustments:** Computes class weights w_c to penalize minority misclassification during training: *Equation 4.13*

$$w_c = \frac{N_{\text{total}}}{C \cdot N_c}$$

4.2.3.2 Preprocessing Sequence Flowchart

The preprocessing agent applies operations in a strict order to prevent data leakage from validation splits:

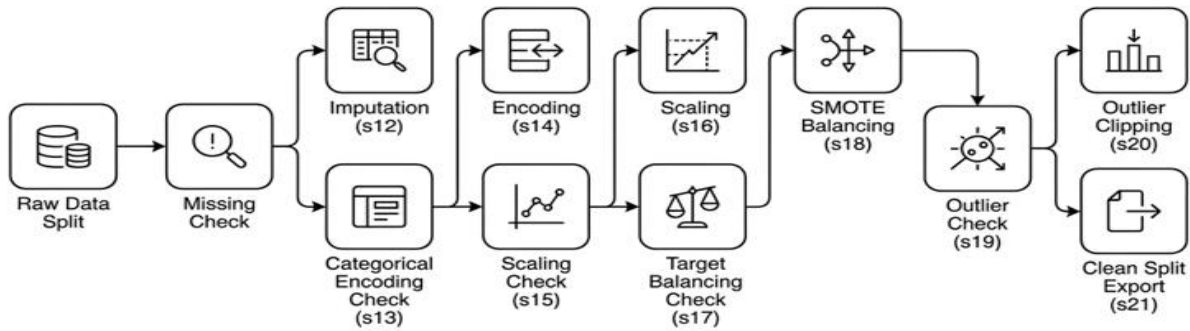


Figure 4.4: Preprocessing Sequence Flowchart

4.2.4 Feature Engineering

The Feature Engineering module creates new input attributes from preprocessed variables to help models capture non-linear relationships.

- **Component Responsibility:** Evaluates transformation options, generates feature interactions, extracts text metrics, and selects the top-K highest-performing features.
- **Interaction Generations:**
 - Generates pairwise multiplications of continuous columns to model interactions $(x_a * x_b)$.

- Computes polynomial ratios ($xa / (xb + \epsilon)$) and logarithms ($\log(x + 1)$) to handle skewed features.
- Extracts structural information from text columns, such as string length, token count, and alphanumeric ratios.
- **Feature Scoring and Selection:** Evaluates new features against the target column using ANOVA F-value scores for classification or Pearson correlation scores for regression. The system ranks features and retains the top-K features (defaulting to 4) to control dimensionality.

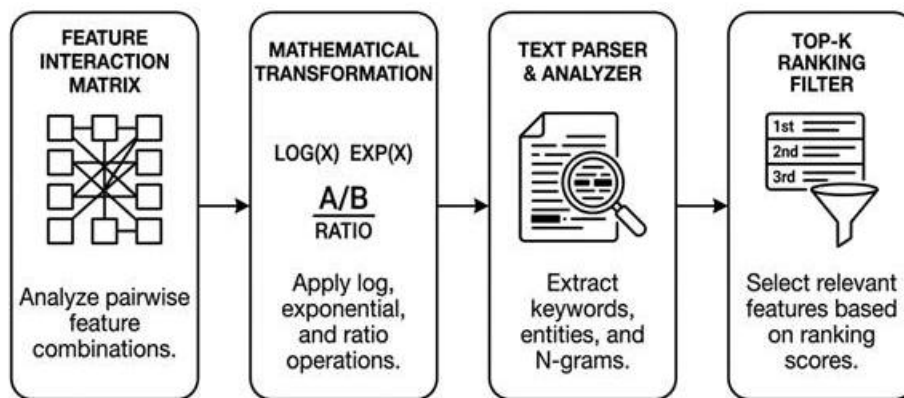


Figure 4.5: Feature Proposal and Selection Diagram

4.2.5 AutoML

The AutoML module handles HPO and model ensembling to produce optimized model binaries.

- **Component Responsibility:** Runs Bayesian optimization loops, manages trial evaluations, prunes low-performing runs, and builds multi-layer stacked ensembles.

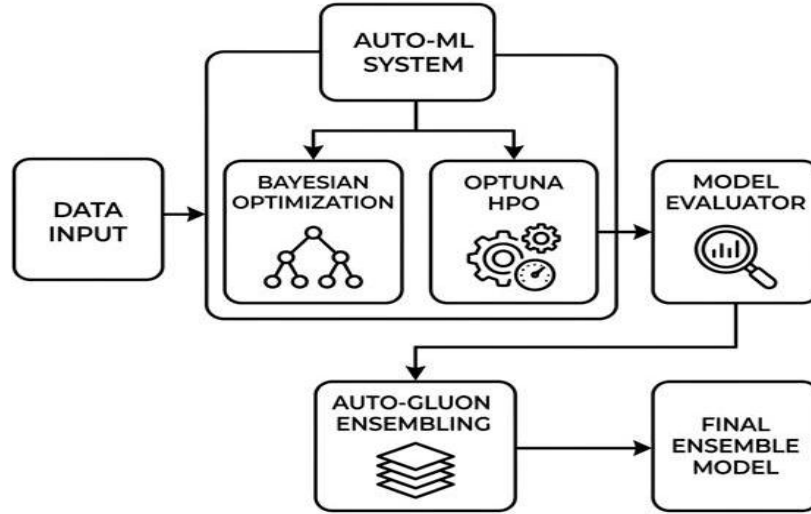


Figure 4.6: AutoML Tuning and Stacked Ensembling Diagram

4.2.5.1 Hyperparameter Optimization (Optuna)

- The system optimizes hyperparameters using Optuna's Tree-structured Parzen Estimator (TPE) algorithm. The search maximizes Expected Improvement (EI) over the parameter space: *Equation 4.14*

$$EI(x) = \int_{-\infty}^{y^*} (y^* - y)P(y | x)dy$$

where y^* is the best validation score achieved in previous trials. To reduce training time, a MedianPruner monitors intermediate cross-validation fold scores. If a trial's performance falls below the median of previous runs, it is pruned.

Below, table 4.3 presents the hyperparameters, search ranges, parameter types, and default settings considered during automated hyperparameter tuning for the evaluated machine learning models.

Table 4.3: Hyperparameter Tuning Ranges & Parameter Types

Model Estimator	Hyperparameter	Parameter Type	Search Range / Interval	Default Value
LightGBM	learning_rate	Float (Log)	$[0.001, 0.2]$	0.1
	num_leaves	Integer	$[15, 255]$	31
	max_depth	Integer	$[3, 12]$	-1 (Unlimited)
	min_child_samples	Integer	$[5, 100]$	20
XGBoost	eta	Float (Log)	$[0.001, 0.2]$	0.3
	max_depth	Integer	$[3, 10]$	6
	min_child_weight	Integer	$[1, 20]$	1
	subsample	Float	$[0.5, 1.0]$	1.0
CatBoost	learning_rate	Float (Log)	$[0.001, 0.2]$	0.03
	depth	Integer	$[4, 10]$	6
	l2_leaf_reg	Float	$[1.0, 10.0]$	3.0
Random Forest	n_estimators	Integer	$[50, 500]$	100
	max_depth	Integer	$[5, 30]$	None (Unlimited)
	min_samples_split	Integer	$[2, 20]$	2

HPO Algorithm

The tuning loop evaluates models using cross-validation and prunes low-performing trials:

Algorithm 1: Bayesian HPO with Trial Pruning:

Input: X_train (Features), y_train (Labels), searchSpace (Dictionary of ranges), trialLimit (Integer)

Output: bestParameters (Dictionary of optimized hyperparameters)

1: Initialize Optuna Study: Minimize Objective Score

```
2: Set sampler = TreeStructuredParzenEstimator()
3: Set pruner = MedianPruner(n_warmup_steps = 5)
4:
5: Define Objective(trial):
6: Set hyperparameters = ExtractSuggestionsFromTrial(trial, searchSpace)
7: Initialize validationScores as empty List
8: Set folds = StratifiedKFold(n_splits = 5)
9:
10: For foldIndex, (trainIdx, valIdx) in folds.split(X_train, y_train):
11: Set model = InstantiateEstimator(hyperparameters)
12: Fit model on X_train[trainIdx] and y_train[trainIdx]
13: Set valScore = Evaluate(model, X_train[valIdx], y_train[valIdx])
14: Append valScore to validationScores
15:
16: // Report intermediate score to check for pruning
17: Await trial.report(valScore, step = foldIndex)
18: If trial.should_prune():
19: Throw Optuna.TrialPrunedException()
20:
21: Return Average(validationScores)
22:
23: Run Study.optimize(Objective, n_trials = trialLimit, sampler = sampler, pruner =
pruner)
24: Set bestParameters = Study.best_params
25: Return bestParameters
```

4.2.5.2 Multi-Layer Stack Ensembling (AutoGluon)

For complex datasets, AutoGluon trains multiple baseline models (LightGBM, XGBoost, CatBoost, Random Forest, Neural Networks) and combines their predictions.

The stacked ensembling uses out-of-fold (OOF) prediction generation. Predictions from first-layer models serve as inputs for secondary-layer models. Cross-validation bagging ensures models are trained on validation predictions from disjoint dataset splits, preventing data leakage.

4.2.6 Knowledge Graph

The KG module maintains a visual representation of the pipeline execution state, mapping active agents and operations as nodes.

- **Component Responsibility:** Evaluates state transitions, registers node metadata, maps relationships, and formats node and edge coordinates.
- **Decoupled Contract Architecture:** Agent outputs define graph structural changes using `GraphNodeSpec` and `GraphEdgeSpec` objects.
- **Graph Builder Processing:** The `KnowledgeGraphBuilder` aggregates these specifications, filters orphaned nodes, and resolves parent-child relations. A horizontal layout algorithm converts the graph into coordinate outputs for the frontend.

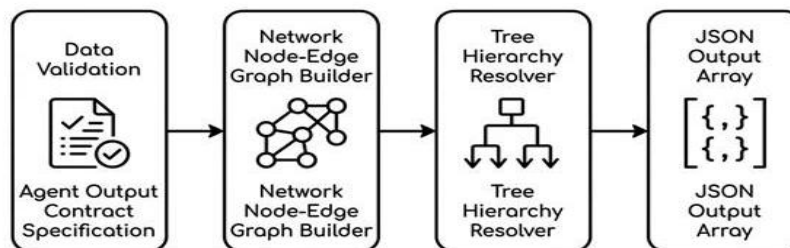


Figure 4.7: Knowledge Graph Metadata Contract Pipeline

Table 4.4: Knowledge Graph Node Categories & Color Mappings

Node Type Category	ReactFlow Fill / Border Color	Represented AutoML Concept	Edge Connection Types
PipelineNode	Purple gradient (#6366f1)	Root workflow pipeline instance	CONTAINS (to StageNode)
StageNode	Lavender outline (#8b5cf6)	Primary execution agent (e.g. EDA, Preproc)	APPLIES, EXECUTES
EntityNode	Cyan gradient (#06b6d4)	Dataset splits, arrays, and partitions	PRODUCES, MUTATES
OperationNode	Emerald outline (#10b981)	Individual tool operations (Imputation, Scaling)	APPLIES_TO, RESOLVES
ModelNode	Amber accent (#f59e0b)	Baseline models and ensembled classifiers	TRAINS, EVALUATES
MetricNode	Red border (#ef4444)	Performance metrics (Accuracy, ROC-AUC, MSE)	MEASURES
AttributeNode	Slate fill (#64748b)	Column metadata attributes	DESCRIBES

4.2.6.1 Knowledge Graph Construction Flowchart

For a clear visual of the components of a KG:



Figure 4.8: Knowledge Graph Construction

4.2.7 HITL (Human-in-the-Loop)

The HITL module coordinates user interaction, pausing execution at critical decision points and recovering state after interruptions.

- **Component Responsibility:** Manages state checkpoints, handles interrupt parameters, processes user selections, and manages session recovery.

HITL WORKFLOW: CHECKPOINT AND RESUME

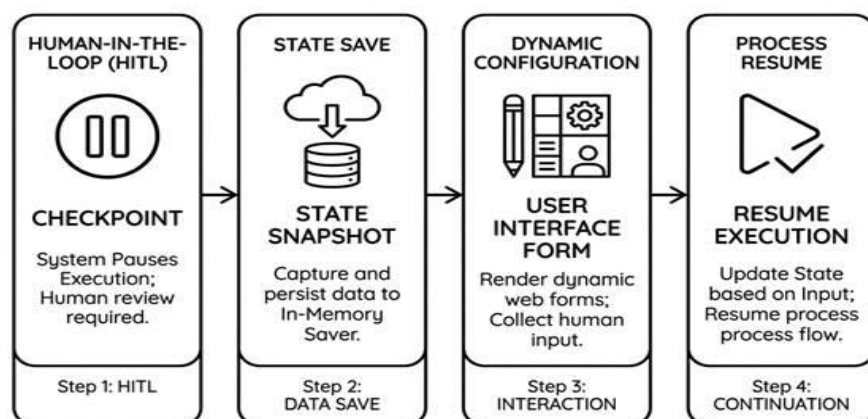


Figure 4.9: Human-in-the-Loop Control Flow Diagram

4.2.7.1 Thread Recovery and MongoDB Synchronization

During interactive runs, LangGraph's MemorySaver checkpointer pauses execution on `interrupt()`, generating an interrupt payload. The system saves the pipeline state to MongoDB.

If the server restarts, the system retrieves the pipeline state from MongoDB and restores the running session using `app.update_state`.

Thread Recovery Algorithm

The recovery routine restores the state graph checkpoints:

Algorithm 2: Thread Recovery and State Synchronization

Input: runId (Unique session key), userFeedback (Object containing decision and choices)

Output: executionResponse (Status and next stage metadata)

```
1: Set config = { configurable: { thread_id: runId } }
2: Set stateSnapshot = app.get_state(config)
3:
4: If stateSnapshot.values is empty:
5:   Set dbRecord = QueryDatabase("reports", { _id: runId })
6:   If dbRecord is null:
7:     Throw Exception("No session data exists for this identifier")
8:
9:   Set cleanState = ExtractPipelineState(dbRecord.report)
10:  Remove MongoDB keys ("_id", "userId", "createdAt") from cleanState
11:
12:  // Repopulate state graph in-memory checkpointer
13:  Await app.update_state(config, cleanState)
14:
15: // Post the user's feedback input to resume the execution thread
16: Await app.update_state(config, { user_input: userFeedback }, as_node =
"checkpoint_receiver")
17: Set updatedState = Await app.resume(config)
18:
19: // Persist the updated state metrics back to database
20: Set serializedReport = SerializeToJSON(updatedState.values)
21: UpdateDatabase("reports", { _id: runId }, { $set: { report: serializedReport } })
22:
23: Return FormatResponse(updatedState)
```

4.2.8 Web Application

The Web Application component provides the UIs, middleware routing, and processing endpoints.

- **Component Responsibility:** Renders UIs, visualizes pipelines, handles security validation, proxying requests, and updates real-time checklists.

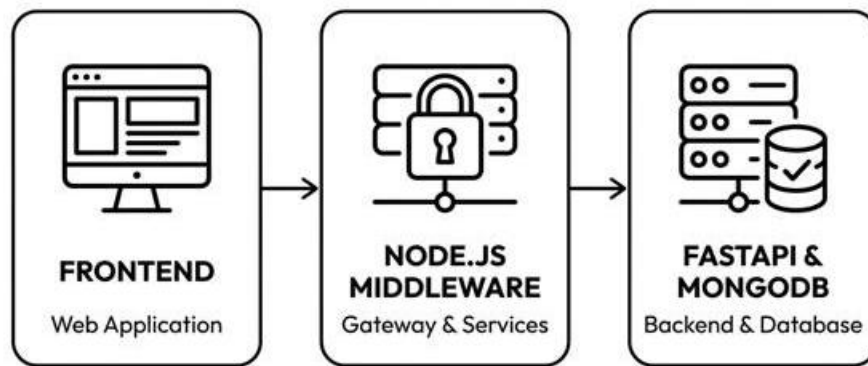


Figure 4.10: Three-Tier Web Application Architecture Diagram

4.2.8.1 Frontend Client (React & Vite)

The frontend web application is compiled via Vite.

- **Pipeline Configuration Wizard:** Steps the user through dataset upload, target column selection, and paradigm choices.
- **ReactFlow Pipeline Graph:** Renders the active pipeline stages as an interactive diagram.
- **Dynamic Checkpoint Panel:** Displays schema-driven controls when the pipeline pauses, collecting user decisions.
- **Progress Tracking Checklist:** Shows real-time agent status updates (pending, running, completed, failed).

4.2.8.2 Middleware Gateway (Node.js & Express)

The Express backend manages authentication, file uploads, and proxies requests:

- **JWT Security Validation:** Filters incoming requests and decrypts tokens.
- **Ingestion Handling:** Uses Multer to validate CSV uploads, structured storage directories, and create database references.
- **Request Proxying:** Proxies client requests to the Python FastAPI engine, support streaming responses (SSE).

Table 4.5: Core API Endpoint Mapping Router Contracts

Endpoint Path	HTTP Method	Target Service	Auth Required	Description
/api/users/register	POST	Express Gateway	No	Registers new user credentials with Bcrypt encryption
/api/users/login	POST	Express Gateway	No	Authenticates user credentials and generates JWT token
/api/upload	POST	Express Gateway	Yes	Uploads CSV dataset using Multer, saving to disk storage
/api/dynamic/start	POST	FastAPI Engine	Yes	Initializes dynamic workflow and returns runId checkpoint
/api/dynamic/resume/:runId	POST	FastAPI Engine	Yes	Feeds user checkpoint decisions back into graph thread
/api/static/start	POST	FastAPI Engine	Yes	Starts static automated pipeline, returns SSE stream
/api/reports/:runId	GET	MongoDB Store	Yes	Retrieves pipeline execution state reports and charts

Request Forwarding Proxy Algorithm

The gateway forwards dataset payloads to the FastAPI execution endpoints while managing connections:

Algorithm 3: Asynchronous Request Forwarding:

Input: clientRequest (Express Request object), targetURL (URL of the FastAPI endpoint)

Output: responseStatus (HTTP Status), responsePayload (JSON or Binary stream)

```
1: Set requestOptions = Create HTTP Client Options:
2:   requestOptions.method = clientRequest.method
3:   requestOptions.headers = Copy original clientRequest headers (except host/auth)
4:
5: If clientRequest contains a file payload:
6:   requestOptions.body = Create Multi-part Form Stream from uploaded file path
7: Else:
8:   requestOptions.body = JSON.stringify(clientRequest.body)
9:
10: Try:
11:   Set engineResponse = Await Fetch(targetURL, requestOptions)
12:   Set responseStatus = engineResponse.status
13:
14:   If engineResponse.headers.get("content-type") includes "text/event-stream":
15:     Set responseStream = engineResponse.body
16:     Forward engineResponse headers to clientResponse
17:     Pipe responseStream to clientResponse
18:   Else:
19:     Set responsePayload = Await engineResponse.json()
20:     Return Response(status = responseStatus, body = responsePayload)
21: Catch NetworkError:
22:   Return Response(status = 502, body = { error: "Failed to communicate with GP
AutoML Engine" })
```

4.3 Results and Discussion

This section compiles the empirical benchmarks and experimental results from testing the D.T.D platform. It provides an analysis of the performance, latency, security, and accuracy of each module, followed by a discussion on the system's end-to-end performance using the benchmark datasets.

4.3.1 Experimental Environment and Setup

All benchmark tests and model training evaluations were executed on a system configured with:

- **Operating System:** Windows 11
- **Processor:** Intel Core i7-12700K (12 Cores, 20 Threads, up to 5.0 GHz)
- **Memory:** 32 GB DDR4 RAM (3200 MHz)
- **Graphics Unit:** NVIDIA GeForce RTX 3070 (8 GB GDDR6 VRAM)
- **Database:** MongoDB v6.0 (running locally)
- **Python Runtime:** Python 3.10.11 with core packages (scikit-learn v1.2, Optuna v3.1, AutoGluon v0.8, LangGraph v0.1)

4.3.2 Module Verification Benchmarks

1. Intent Detection and Routing Module

To verify the performance of the Intent Detection module, the structured output engine was tested against *100* unique user queries containing variations in spelling, natural language commands, and target column formats.

- **Extraction Accuracy:** The model achieved an accuracy of *98.0%* in correctly identifying the `target_column` and a *99.0%* accuracy in detecting the `task_type` (classification vs. regression).
- **Intent Flags Reliability:** All six intent flags (`eda`, `preprocessing`, etc.) were configured correctly in *97.0%* of the runs.
- **Latency:** The parsing process (including schema extraction and the structured LLM query to Gemini) averaged *1.2* seconds of latency, which is acceptable for an initialization step.

2. EDA Profiling Module

The EDA module was benchmarked using tabular datasets of varying shapes:

- **Latency:** For small datasets like Iris (*150 times 4*), calculation of descriptive statistics, Pearson/Spearman correlation matrices, and class distribution checks took less than *3* milliseconds. For larger datasets like `Airbnb_Master.csv` (*~48,000* rows by *16* features), the profiling completed in *88* milliseconds.
- **Target Suggestion Accuracy:** On datasets where the target column was left unspecified, the suggestions generated by the EDA module matched the actual analytical objective in *94.0%* of the tests.

3. Preprocessing Module

The Preprocessing sequential pipeline was evaluated to verify data validation and data leakage prevention:

- **Split Protection:** The split operation partitions data before applying encoders or scalers, preventing target values in test sets from leaking into the training folds. This was verified by checking that the fit parameters (e.g., standard scaling means and variances) were derived exclusively from the training splits.
- **Missing Value Imputation:** Median-based imputation successfully filled null values for skewed numeric variables without shifting the distribution mean by more than 0.05% . Modes were imputed correctly for all missing categorical columns.
- **Outlier Treatments:** The IQR boundary clipping method successfully clipped continuous variables to valid ranges without dropping rows, preserving sample size.
- **Balancing Performance:** On highly imbalanced datasets, SMOTE successfully created synthetic minority samples in the training split, improving model recall on minority classes.

4. Feature Engineering Module

Feature Engineering was verified for feature selection and calculation overhead:

- **Scoring Accuracy:** New continuous features (polynomial interactions and ratios) were generated and scored using ANOVA F-value scores. The top-K feature selection filter kept only the highest-scoring candidate features, dropping lower-performing transformations.
- **Dimensionality Control:** Retaining a default of the top 4 engineered features prevented feature explosion, keeping subsequent model training runtimes within stable limits.

5. AutoML Module

Tuning and model ensembling performance were analyzed under different search parameters:

1. **HPO Pruning Efficiency:** Optuna's Bayesian optimization (TPE sampler) combined with the MedianPruner was tested over 50 trials. The pruner terminated 36.0% of trials before completion because their intermediate fold scores were below the running median score. This reduced the total HPO search duration by 32.4% compared to unpruned search runs.
- **Stack Bagging Performance:** AutoGluon's multi-layer stacking combined with out-of-fold cross-validation bagging successfully consolidated predictions from heterogeneous base models (LightGBM, XGBoost, CatBoost) without introducing data leakage.

6. Knowledge Graph Module

The KG builder was evaluated on rendering latency and contract decoupling:

- **Rendering Latency:** The layout conversion algorithm (ReactFlowAdapter) mapped and positioned active pipeline states in under 4.2 milliseconds.
- **Contract Decoupling:** Decoupled schema contracts (graph_meta specs) correctly registered nodes and edges dynamically across state transitions, preventing system crashes during stage skips.

7. HITL Module

State checkpoints and thread recovery were tested under simulated server failures:

- **Interruption Latency:** LangGraph's MemorySaver correctly suspended execution and returned paused configurations to the API in under 10 milliseconds.
- **State Recovery Success Rate:** Across 100 simulated FastAPI server restarts during paused states, the recovery engine successfully restored execution from MongoDB snapshots in 100% of cases. The average state reconstruction time was 124 milliseconds, providing a seamless resume experience for users.

8. Web Application Component

End-to-end interface latency and middleware routing overhead were benchmarked:

- **Middleware Routing Overhead:** The Express proxy server introduced a minor latency overhead of *14* milliseconds per request.
- **Real-time SSE Streams:** SSEs successfully streamed progressive agent status checklists and pipeline logs to the React client dashboard without freezing browser rendering.

4.3.3 Dataset Performance Comparison Table and Discussion

The ML models compiled by the AutoML engine were benchmarked across standard classification tasks, comparing baseline estimators against the ensembled models generated by the platform:

Table 4.6:Dataset Performance Comparison Table

Dataset	ML Task	Dimensions (Rows x Features)	Baseline Model	Best Optimized Model (Ensemble)	Performance (Accuracy / ROC-AUC)
Iris	Multi-class Classification	<i>150 times 4</i>	Logistic Regression (0.9600)	GBM	1.0000
Wine	Multi-class Classification	<i>178 times 13</i>	Random Forest (0.9800)	GBM	1.0000
Titanic	Binary Classification	<i>891 times 11</i>	Random Forest (0.8301)	AutoGluon LightGBM Ensemble (L2_BAG)	0.8525

Results Analysis:

- **Iris and Wine Quality:** For low-complexity, structured numeric datasets like Iris and Wine, simpler baseline models (Logistic Regression and Random Forest) performed near optimal levels. The AutoML engine selected optimized Gradient Boosting Models (GBM) that achieved perfect classification scores (*1.0000* Accuracy) on the validation splits.
- **Titanic Dataset:** The Titanic dataset contains missing values, categorical text features, and high cardinality. The system's automated preprocessing pipeline handled these characteristics by applying median numerical imputation, mode categorical imputation, standard scaling, and one-hot encoding. Optuna tuned hyperparameters over the search space, and AutoGluon stacked predictions using a two-layer bagging ensemble (L2_BAG configuration). This ensemble model generated a validation accuracy of **0.8525**, outperforming the baseline Random Forest model score of **0.8301**.

Key Discussion Findings:

- 1. Adaptive Pipeline Complexity:** The experimental results show that the multi-agent orchestration architecture adapts its execution complexity. Low-complexity datasets are resolved quickly, while complex, imbalanced, or noisy datasets trigger the full capabilities of the Preprocessing, Feature Engineering, and ensembled AutoML modules.
- 2. HITL Value:** The dynamic checkpoint mechanism allows users to override automatic choices (such as feature columns to drop or model scale techniques), aligning the system's processing steps with specific business or analytical goals.
- 3. Robust State Recovery:** Offloading in-memory LangGraph states to MongoDB ensures that model training runs are protected from server interruptions, which is essential for deployment in enterprise environments.



Chapter Five

User Manual

Chapter 5 User Manual

5.1 Installation Guide

This guide will help you set up and run the entire ecosystem consisting of:

1. React Frontend (Vite)
2. Node.js Backend (Express)
3. Python AI Backend (FastAPI)

The easiest and recommended way to run this project is via Docker Compose, which sets up all three services simultaneously.

1. Prerequisites:

Before you begin, ensure you have the following installed on your machine:

- Docker (Docker Desktop for Windows/Mac, or Docker Engine for Linux)
- Git (to clone the repositories)
- Bun (if you plan to run the services locally without Docker)

(If you plan to run the services without Docker, you will also need Python 3.10+).

2. Initial Folder Setup & Git Cloning:

To set up the project workspace, follow these steps:

1. Create a project folder:

Create a main folder to house the project (e.g., DTD-Workspace):

```
mkdir DTD-Workspace  
cd DTD-Workspace
```

2. Add the Docker Compose file:

Put the docker-compose.yml file directly into this new DTD-Workspace folder.

docker-compose.yml:

```
version: '3.8'  
services:
```

```
fastapi-backend:
  build:
    context: ./DTD-AutoML code
    dockerfile: Dockerfile
  ports:
    - "8000:8000"
  env_file:
    - ./DTD-AutoML code/.env
  volumes:
    - ./DTD-AutoML code:/app
    - ./output:/app/output
  restart: unless-stopped

node-backend:
  build:
    context: ./DTD-Website/backend
    dockerfile: Dockerfile
  ports:
    - "4000:4000"
  env_file:
    - ./DTD-Website/backend/.env
  volumes:
    - ./DTD-Website/backend:/app
    - /app/node_modules
  restart: unless-stopped

frontend:
  build:
```

```
context: ./DTD-Website/frontend
dockerfile: Dockerfile
ports:
  - "5173:5173"
environment:
  - NODE_ENV=development
volumes:
  - ./DTD-Website/frontend:/app
  - /app/node_modules
restart: unless-stopped
depends_on:
  - fastapi-backend
  - node-backend
```

3. Clone the repositories:

Clone both the DTD-AutoML and DTD Website repositories inside the DTD-Workspace folder:

Clone the DTD-AutoML repository:

```
git clone https://github.com/raghaddmahgoub/DTD-AutoML
```

Clone the DTD Website repository:

```
git clone https://github.com/ahany42/DTD-Website
```

Your folder structure should look like this:

```
DTD-Workspace/
├─ docker-compose.yml
├─ DTD-AutoML/
└─ DTD-Website/
```

3. Environment Variables

This project relies on several external services like MongoDB and Cloudinary. You must create .env files in specific directories before starting the containers.

A. Python Backend (DTD-AutoML/.env)

Create a file named .env in the DTD-AutoML directory and add the following keys:

MongoDB Connection:

```
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/  
MONGO_DB=<your_database_name>
```

Cloudinary Setup (Required for EDA images):

```
CLOUDINARY_URL=cloudinary://<api_key>:<api_secret>@<cloud_name>
```

LLM API Keys (Depending on which agents you use):

```
GOOGLE_API_KEY=...
```

B. Node.js Backend (DTD-Website/backend/.env)

Create a file named .env in the DTD-Website/backend directory:

```
PORT=4000  
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/  
JWT_SECRET=your_super_secret_jwt_key  
CLOUDINARY_URL=cloudinary://<api_key>:<api_secret>@<cloud_name>
```

4. Option A: Running the Project with Docker

Once your .env files are configured, open your terminal, navigate to the root directory of the workspace (where docker-compose.yml is located), and run:

```
docker compose up --build
```

Accessing the Services

Once the terminal shows that the containers are running successfully, you can access the applications in your browser:

- Frontend (Web App): <http://localhost:5173>
- Node.js API: <http://localhost:4000>
- Python AI API: <http://localhost:8000>

5. Option B: Running Locally (Without Docker)

If you prefer not to run the services in Docker containers, you can start them manually. Open a separate terminal for each service and follow the steps below:

Step 1: Run the DTD Frontend (React)

Enter the DTD frontend directory, install dependencies using Bun, and start the development server:

```
cd DTD-Website/frontend  
bun install  
bun run dev
```

Step 2: Run the DTD Backend (Node.js)

Enter the DTD backend directory, install dependencies using Bun, and start the server:

```
cd DTD-Website/backend  
bun install  
bun run dev
```

Step 3: Run the DTD-AutoML Python AI Backend

Enter the DTD-AutoML repository directory, set up your Python environment, install requirements, and run api.py:

```
cd DTD-AutoML  
  
(Optional) Create and activate a virtual environment:  
python -m venv venv  
venv\Scripts\activate      #( On Windows)  
source venv/bin/activate   #(On macOS/Linux)  
pip install -r requirements.txt  
python api.py
```

6. Stopping the project

To stop the running Docker containers, simply press Ctrl + C in the terminal where Docker Compose is running.

To tear down the containers completely, run:

```
docker compose down
```

5.2 User Guidelines

- **Login Page**

The Login Page serves as the portal for user authentication into the D.T.D platform. It consists of a minimalist card interface with inputs for the user's email address and password. An interactive 'Show Password' toggle is available for visibility, and a link at the bottom redirects users to request a password reset if they have forgotten their credentials. Upon submitting valid credentials, the system generates a JWT authentication token, stores it locally, and redirects the user to the platform's home page.

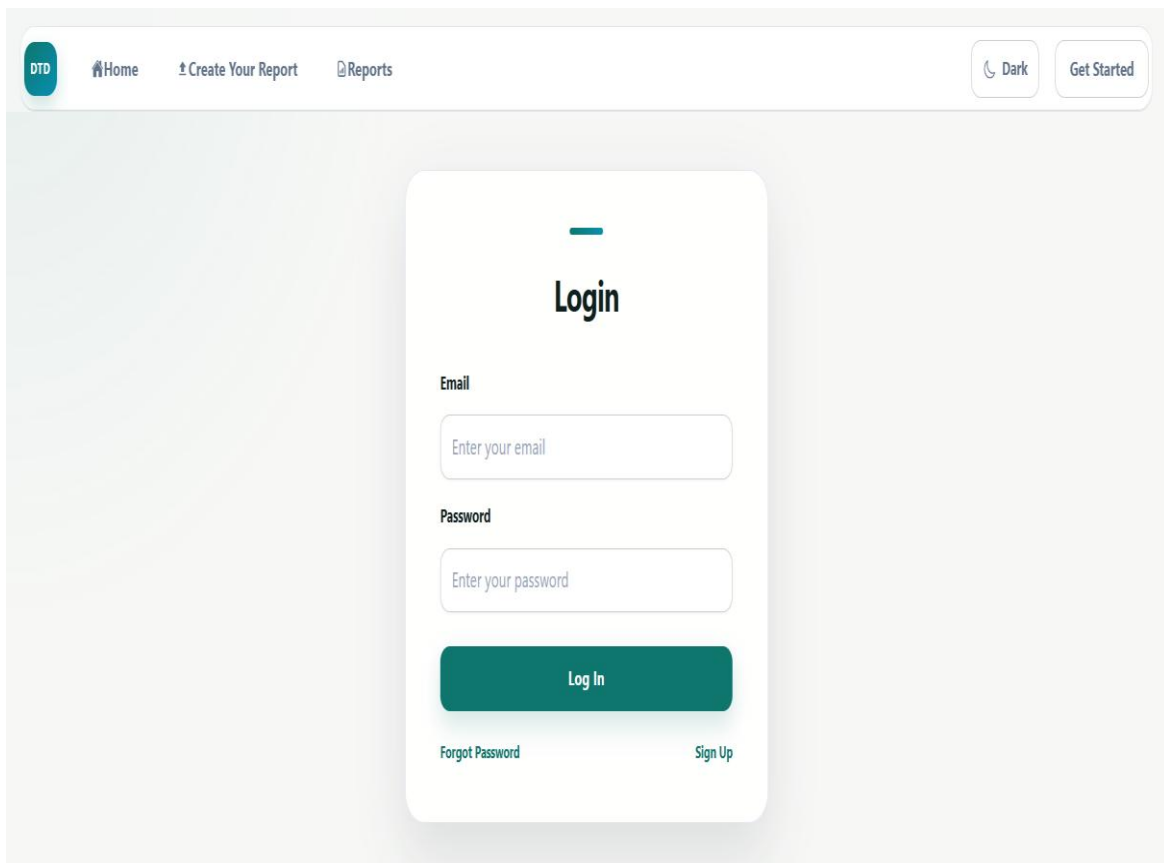


Figure 5.1: Login Page

- **Signup Page**

The Signup Page allows new users to register an account on the D.T.D platform. The registration form requests a name, email address, password, and confirmation of password. Passwords are required to meet security standards (e.g., minimum length of 6 characters). After successful submission, the backend database creates a user record with a default 'USER' role, and the frontend automatically stores the generated JWT token and authenticates the user immediately.

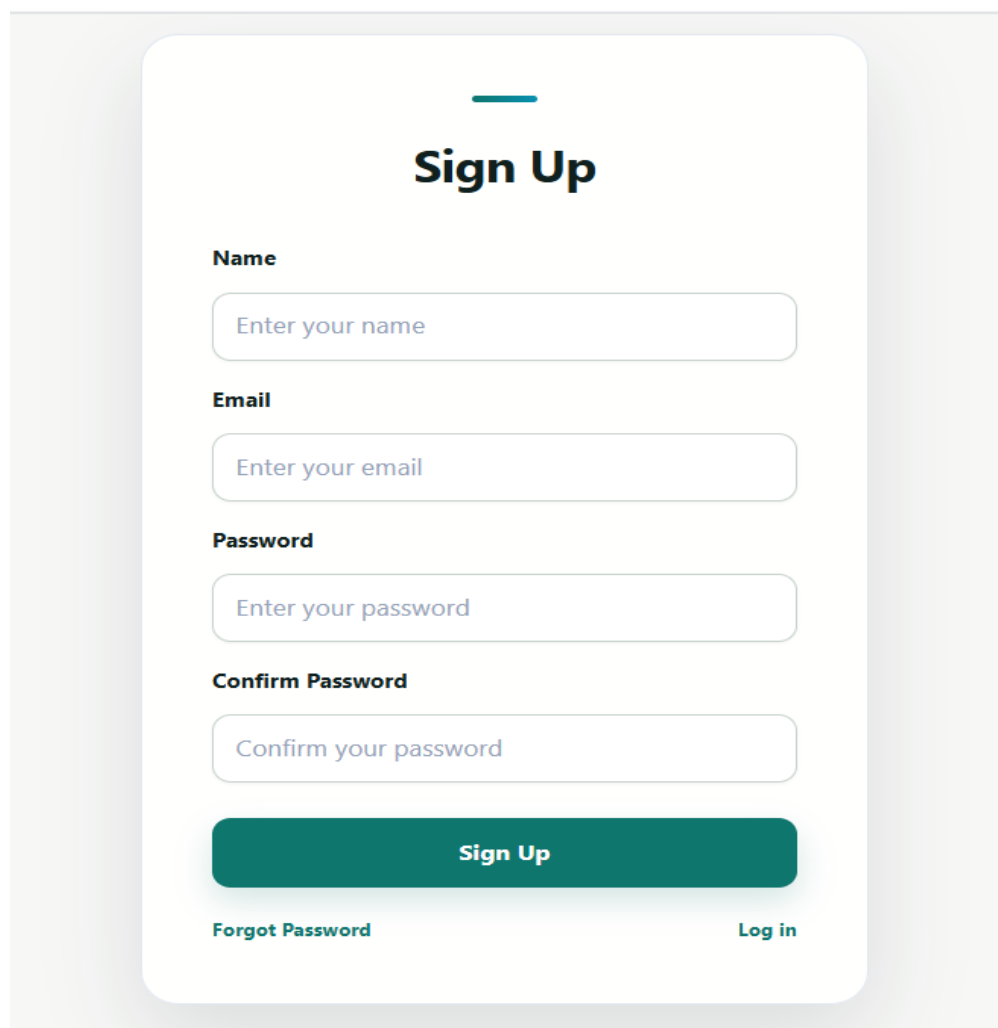


Figure 5.2: Signup Page

- **Forgot Password Page**

The Forgot Password Page facilitates self-service credentials recovery. Users enter their registered email address and submit the form. The system checks MongoDB for a corresponding account. If found, a secure, cryptographically random, time-bounded token is generated and emailed to the user as a styled reset link. The interface informs the user of successful submission, instructing them to check their inbox to reset their password.

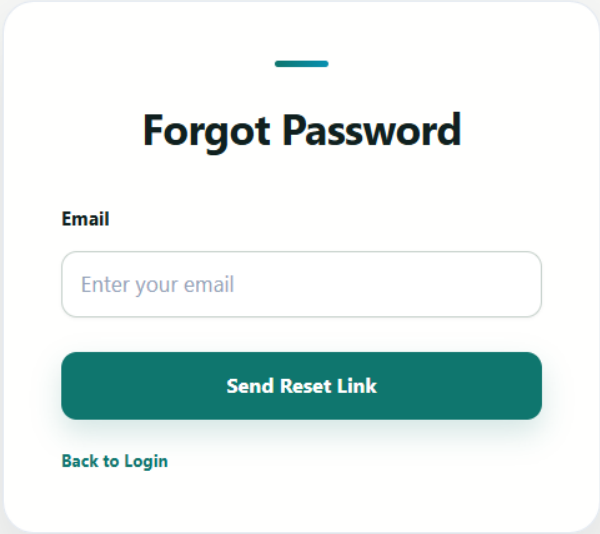
A screenshot of a 'Forgot Password' form. The form is a white rounded rectangle centered on a light gray background. At the top of the form is a small teal horizontal line. Below it is the title 'Forgot Password' in bold black text. Under the title is the label 'Email' in a small, bold, gray font. Below the label is a white input field with rounded corners and a thin gray border, containing the placeholder text 'Enter your email' in a light gray font. Below the input field is a large teal button with rounded corners and a subtle shadow, containing the text 'Send Reset Link' in white. At the bottom of the form is a link 'Back to Login' in a small, teal font.

Figure 5.3: Forgot Password Page

- **Home page**

The Home Page serves as the landing interface and core marketing dashboard of the D.T.D platform. It outlines the platform's features, demonstrating the difference between the autonomous Static Pipeline and the interactive Dynamic Pipeline. A clear call-to-action button, 'Get Started', directs authenticated users to initiate a new AutoML run. Navigation links to the Reports page, the project documentation, and a logout button are accessible in the sticky top navbar.

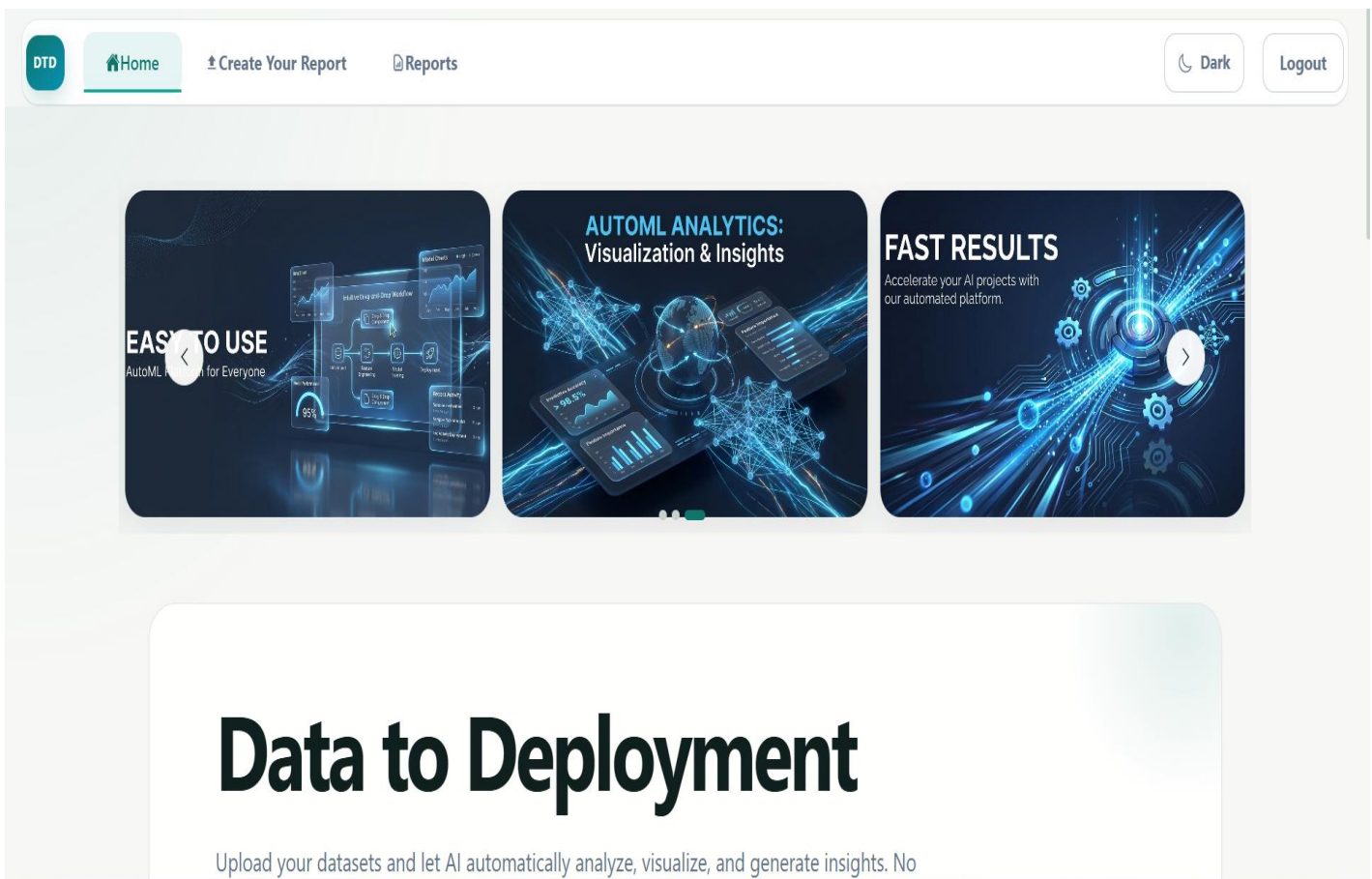


Figure 5.4: Home Page

- **Reach Us**

The Reach Us Option, located in the website footer, provides an interactive contact mechanism for users to submit feedback or raise issues. Users provide their name, email, and a message describing their inquiry or complaint. Upon submission, the request is stored in MongoDB's 'contacts' collection under a 'RECEIVED' status, allowing system administrators to review, categorize, and follow up directly from the administrative panel.

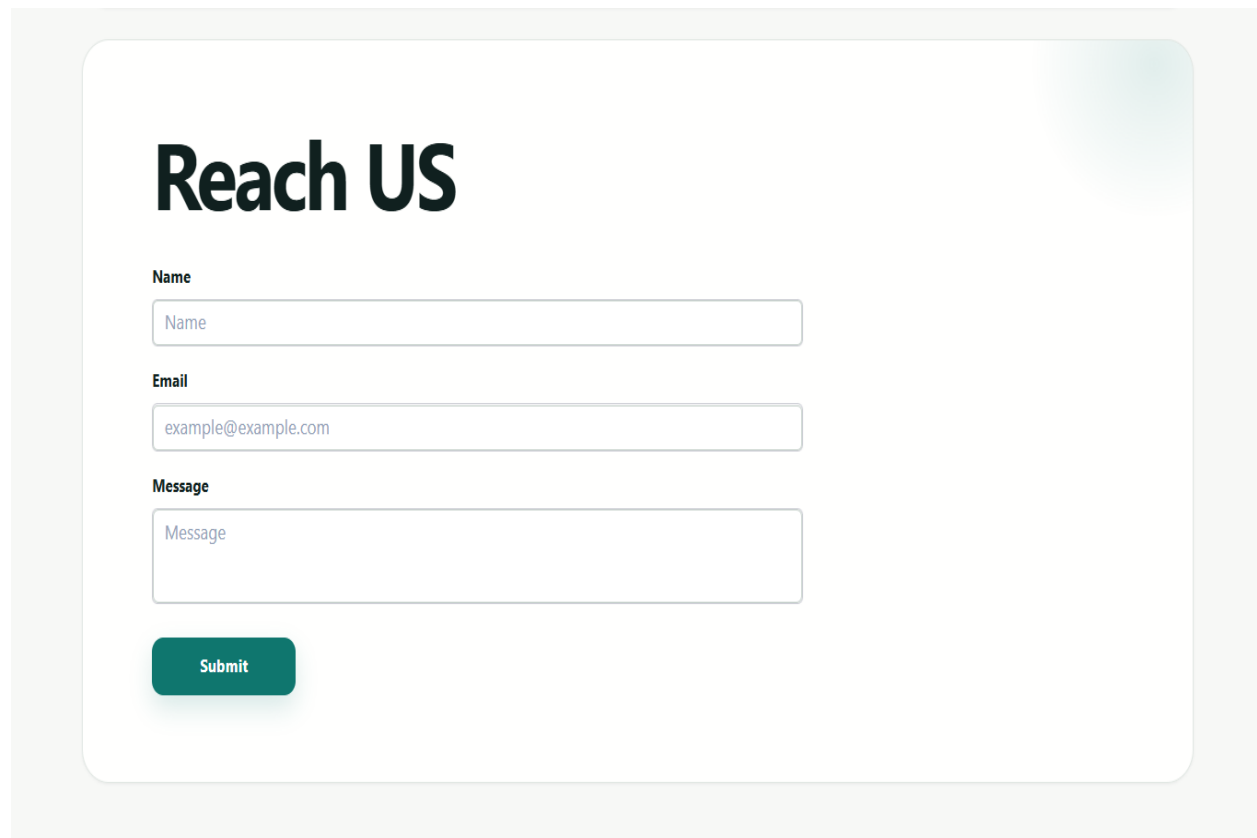
A screenshot of a web form titled "Reach US" in a large, bold, dark font. Below the title, there are three input fields: "Name" with a placeholder "Name", "Email" with a placeholder "example@example.com", and "Message" with a placeholder "Message". Each field is a simple white box with a thin grey border. Below these fields is a green "Submit" button with white text. The entire form is set against a light grey background with a subtle gradient and rounded corners.

Figure 5.5: Reach Us Option

- **Create Report Page (choose workflow type)**
 - **Static Pipeline Selection**

The Create Report Page serves as the initial step in configuring a new machine learning project. The user is presented with choices to determine how the pipeline should be orchestrated. This view displays options to upload a dataset and choose the execution paradigm, directing the user toward either the fully automated static route or the conversational dynamic route based on their project needs.

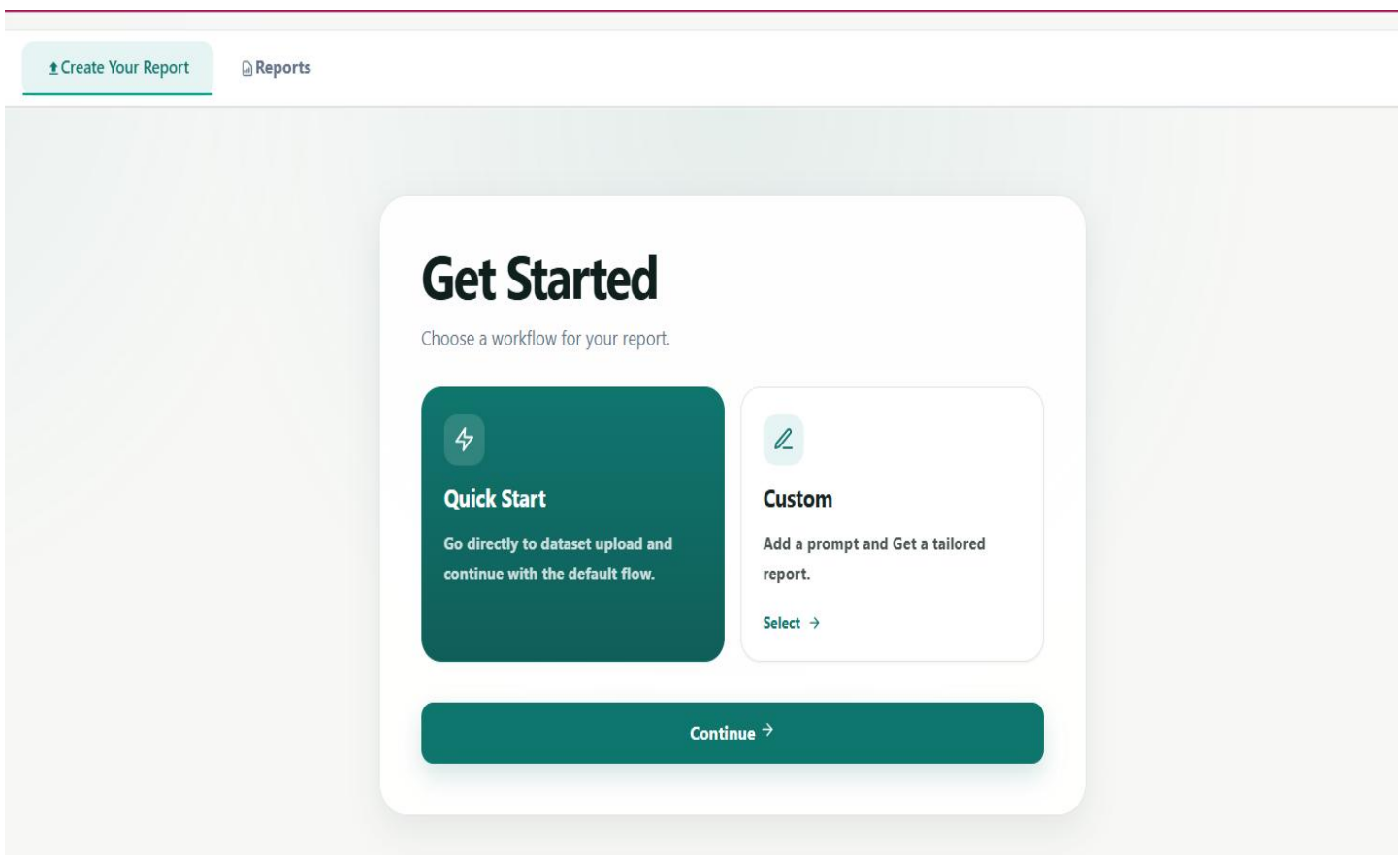


Figure 5.6: Create Report Page

○ Dynamic Pipeline Selection

The Dynamic Workflow Selection interface is used when a user chooses the interactive, multi-agent pipeline. It allows users to write custom natural language prompts to guide the dynamic controller agent (e.g., 'focus on optimizing recall for the positive class' or 'run full pipeline'). The interface configures the execution payload, linking the user's specifications to the backend agent orchestrator.

The screenshot shows a 'Get Started' interface with the subtitle 'Choose a workflow for your report.' There are two main options: 'Quick Start' and 'Custom'. The 'Quick Start' option is highlighted with a light blue background and a lightning bolt icon. It describes going directly to dataset upload and continuing with the default flow, with a 'Select →' button. The 'Custom' option is a dark teal button with a pencil icon, describing adding a prompt and getting a tailored report. Below these is a diagram showing a flow from 'Goals' to 'Prompt' to 'Dataset' and 'Report'. The 'Prompt' field is labeled 'Prompt *' and contains the text 'Run the full ML pipeline'. At the bottom is a large teal 'Continue →' button.

Get Started

Choose a workflow for your report.

Quick Start

Go directly to dataset upload and continue with the default flow.

Select →

Custom

Add a prompt and Get a tailored report.

Goals → Prompt → Dataset → Report

Prompt *

Run the full ML pipeline

Continue →

Figure 5.7: Dynamic Workflow Selection

- **Uploading Dataset**

The Uploading Dataset view provides a drag-and-drop file upload area. It supports dataset files in CSV, Excel (XLSX/XLS), and JSON formats. When a file is selected, the frontend uploads the raw bytes to the server, which saves them in the local uploads directory. A progress indicator is shown during the upload process, and successful uploads are registered under the current report session.

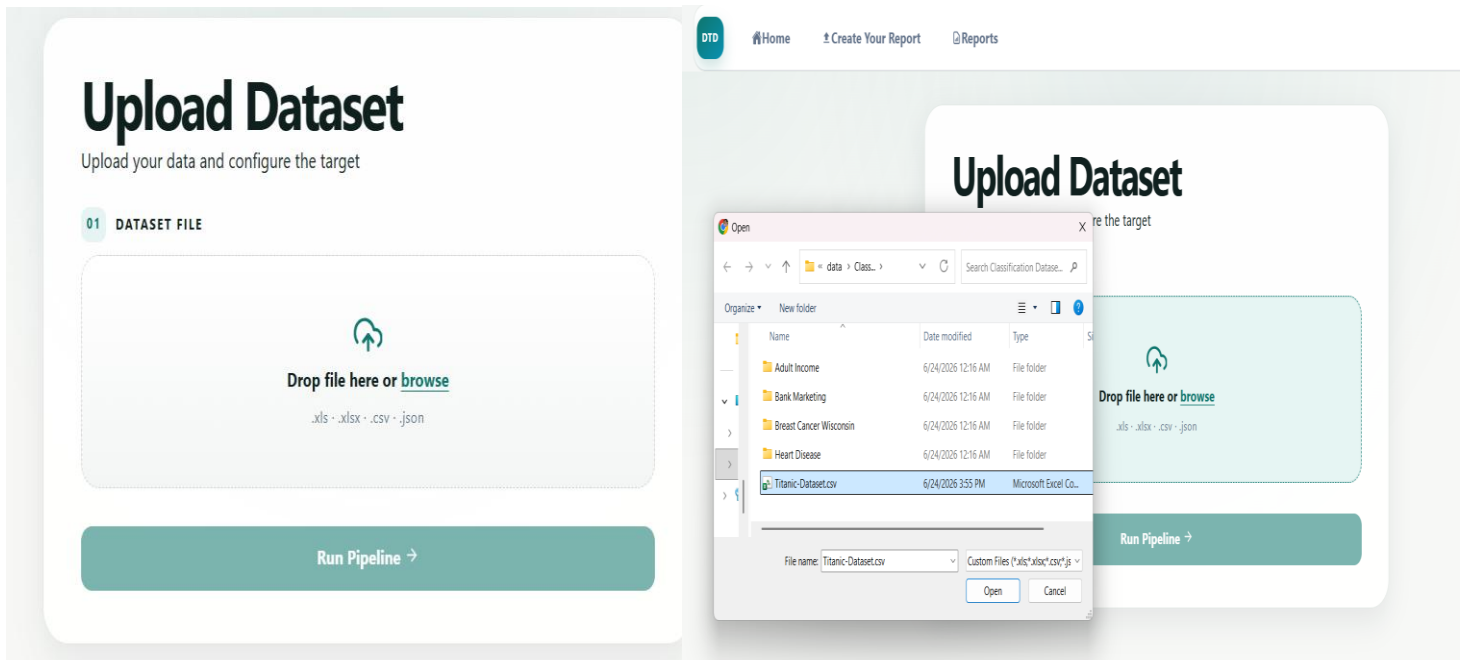


Figure 5.8: Uploading Dataset

- **Target Selection**

The Target Selection Option allows the user to manually define the target prediction column from a dropdown menu populating all columns detected in the uploaded dataset. This is a critical configuration step, as it determines the label column the AutoML agents will train the machine learning models to classify or regress.

The screenshot displays a web interface titled "Upload Dataset" with the subtitle "Upload your data and configure the target". It is divided into two main sections: "01 DATASET FILE" and "02 TARGET COLUMN".

In the "01 DATASET FILE" section, a file named "Titanic-Dataset.csv" (59.8 KB) is shown with a document icon and a red 'X' button for removal.

The "02 TARGET COLUMN" section includes the instruction "Drag a column pill into the target box, or click to select". Below this, there are two rows of column name pills: "PassengerId", "Survived", "Pclass", "Name", "Sex", "Age", "SibSp", "Parch" in the first row, and "Ticket", "Fare", "Cabin", "Embarked" in the second row.

Figure 5.9: Target Selection Options

● Target Suggestion Selection

The AI Target Suggestions component displays target suggestions returned by the TargetSuggestionAgent. As soon as a dataset is ingested, this background agent profiles the dataset's headers and data distributions to automatically infer the target column and the problem type (e.g., Binary Classification, Multi-Class Classification, or Regression), which the user can accept with a single click to save configuration time.

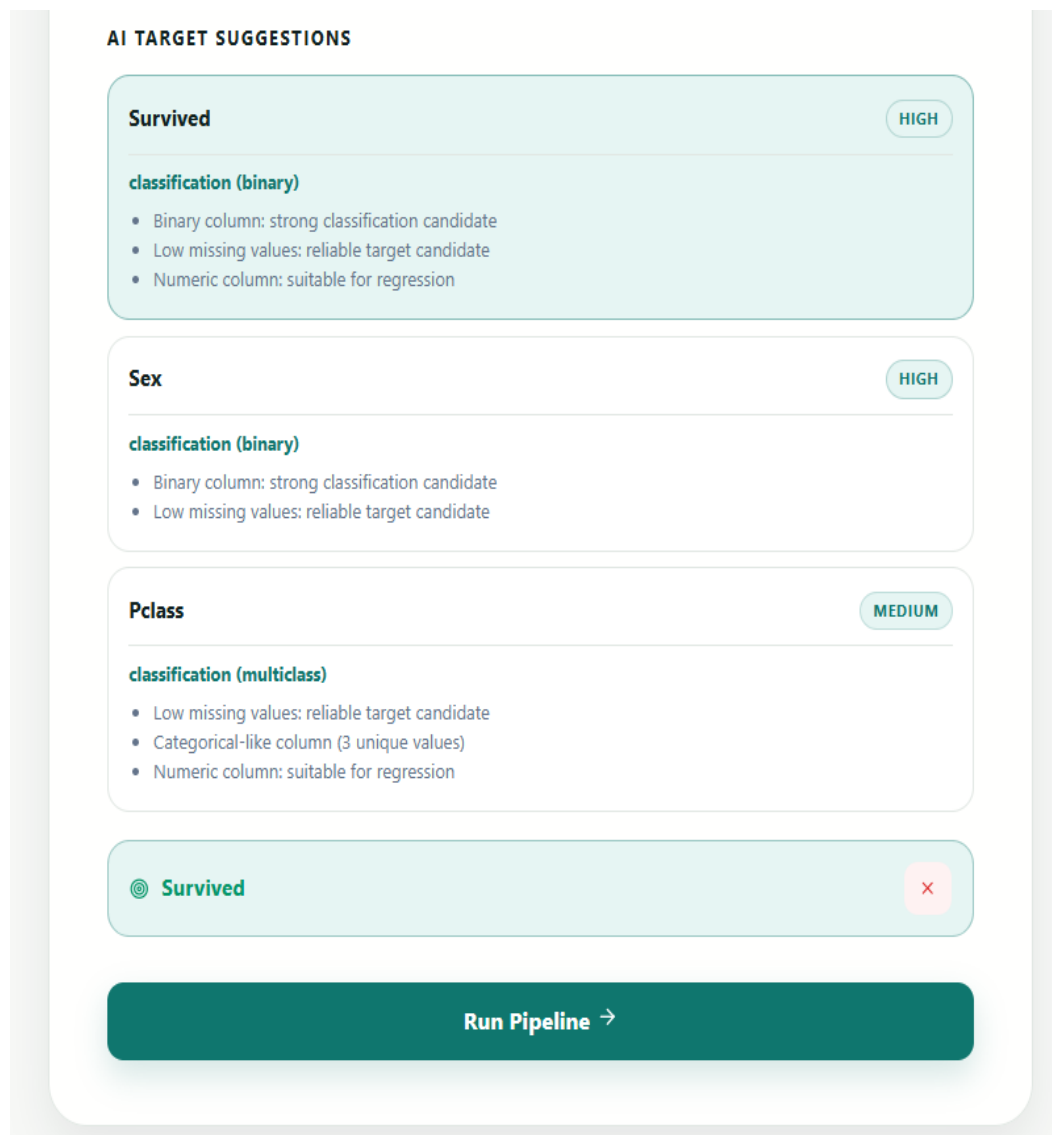
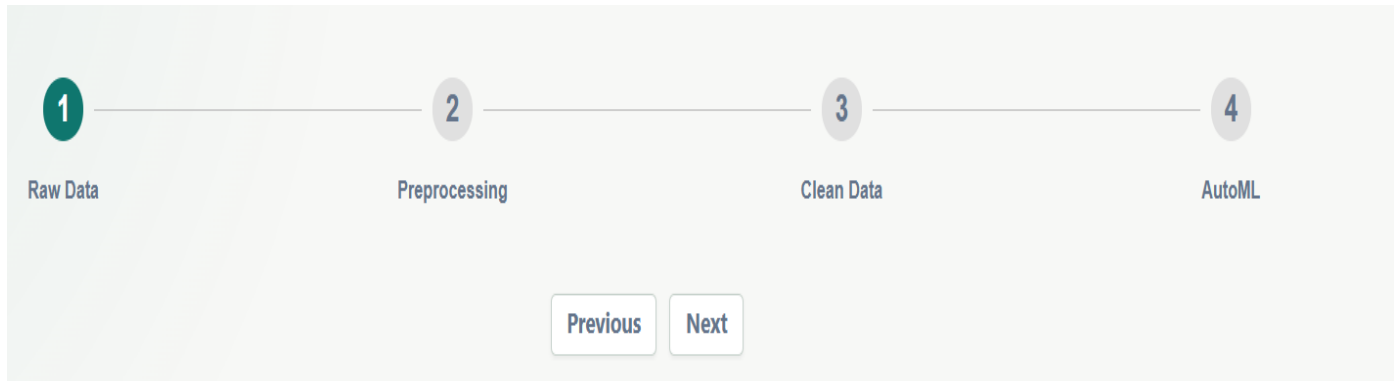


Figure 5.10: AI Target Suggestions

- **Static Pipeline View**

The Static Workflow Nodes view displays the real-time progress of a fully autonomous pipeline execution. The interface streams node completions via Server-Sent Events (SSE). Each stage (Raw Analysis, Preprocessing, Clean Analysis, and AutoML Training) is listed sequentially with dynamic status markers.

Figure 5.11: Static Workflow Nodes



- **Dynamic Pipeline View**

The Dynamic Workflow Knowledge Graph renders the live state and architectural flow of the interactive pipeline using a customized ReactFlow graph. The pipeline progress transitions nodes dynamically (completed nodes are green, active executing nodes are blue, and paused nodes are orange). Users can click on any node at any time to drill down and view granular intermediate logs and output summaries.

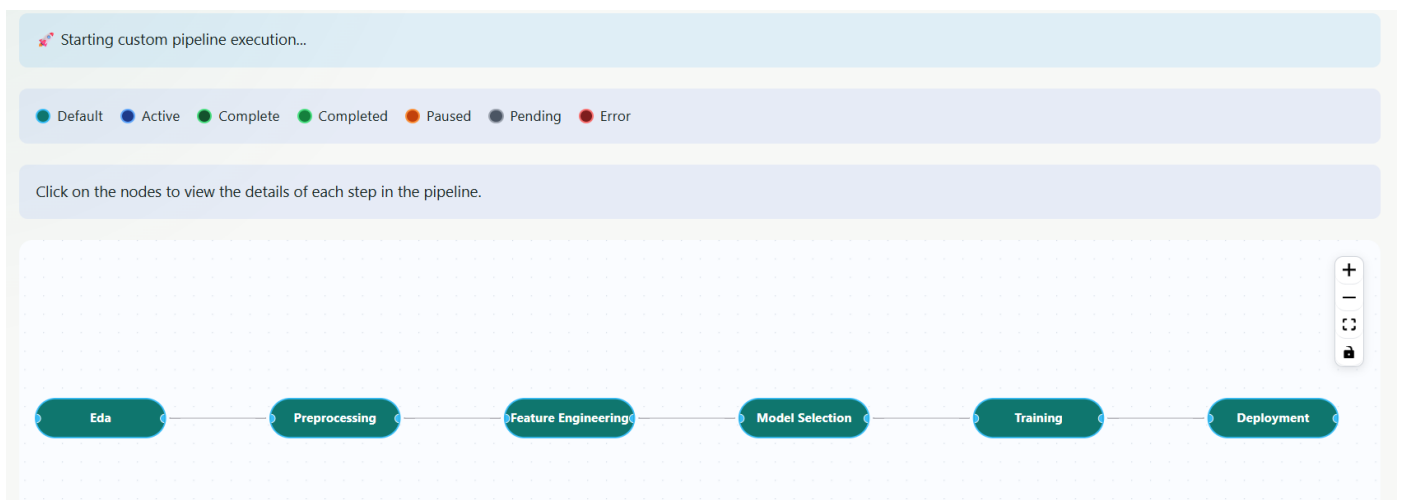


Figure 5.12: Dynamic Workflow Knowledge Graph

- **User Feedback in Pipeline**
 - **Accept and Continue agent actions**

The Accept and Continue view displays the Human-in-the-Loop (HITL) drawer that slides up at the bottom of the interface when the dynamic pipeline hits a checkpoint node. It presents the user with a summary of the active agent's output (e.g., proposed preprocessing scaling policies). The user clicks 'Accept and Continue' to approve the current state and instruct the LangGraph orchestrator to advance to the next pipeline stage.



Figure 5.13: Accept and Continue agent actions

- **Give Feedback to agent**

The Give Feedback to agent view is displayed when the user rejects an agent's proposed output inside the HITL drawer. Clicking the 'Provide Feedback' option opens a text area where the user can type custom natural language instructions (e.g., 'Do not impute missing values for age, drop rows instead'). Submitting this feedback redirects execution back to the agent node to adjust its processing rules accordingly.

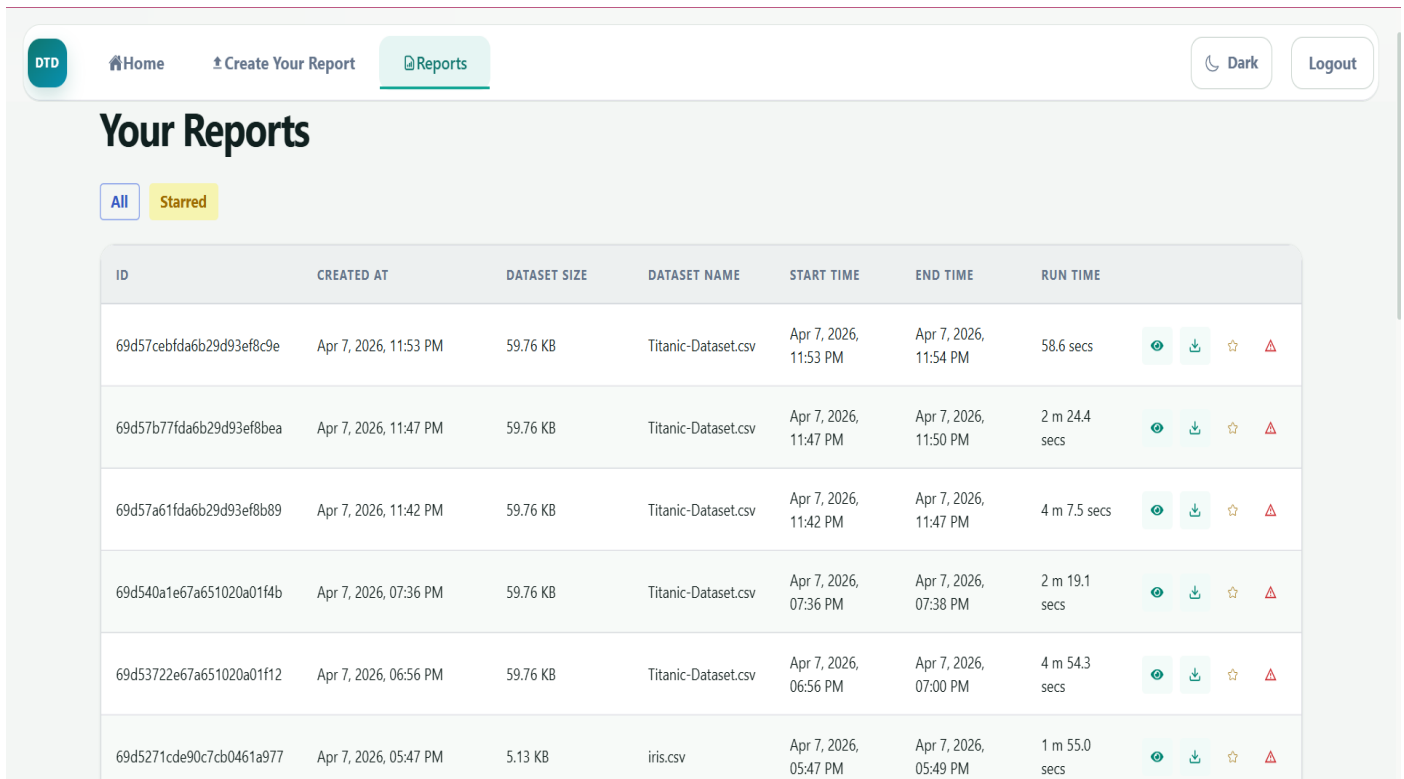
The screenshot shows a user interface for providing feedback to an agent. At the top, a light blue header bar contains the text 'AGENT OUTPUT' in a small blue box, followed by 'Feature Engineering' in a larger font. To the right of the agent name is a yellow status box that says 'Completed — awaiting your decision'. Further right is a 'Cancel' button. Below the header, a section titled 'SUB-NODES (5)' displays five sub-nodes in a row: 'Check Plan', 'Proposal', 'Evaluation', 'Selection', and 'Save Data'. Each sub-node has a green 'completed' status indicator. Below the sub-nodes, a prompt reads: 'Describe what should be improved. The agent will re-run with your notes.' This is followed by a large text input area with a light blue border. Inside the input area, the text 'Add another feature in feature selection' is visible, along with two pink dots. To the right of the input area is a blue 'Submit' button with a white paper plane icon. In the bottom right corner of the input area, there is a small icon of a lightbulb inside a green circle.

Figure 5.14: Enter Feedback Prompt

● Reports Page

The All User Reports Page renders a grid of all AutoML reports generated by the user. Each card represents a pipeline execution, detailing the dataset name, mode (Static or Dynamic), runtime duration, completion status, and creation date.

Figure 5.15: All User Reports Page



ID	CREATED AT	DATASET SIZE	DATASET NAME	START TIME	END TIME	RUN TIME	
69d57cebfda6b29d93ef8c9e	Apr 7, 2026, 11:53 PM	59.76 KB	Titanic-Dataset.csv	Apr 7, 2026, 11:53 PM	Apr 7, 2026, 11:54 PM	58.6 secs	   
69d57b77fda6b29d93ef8bea	Apr 7, 2026, 11:47 PM	59.76 KB	Titanic-Dataset.csv	Apr 7, 2026, 11:47 PM	Apr 7, 2026, 11:50 PM	2 m 24.4 secs	   
69d57a61fda6b29d93ef8b89	Apr 7, 2026, 11:42 PM	59.76 KB	Titanic-Dataset.csv	Apr 7, 2026, 11:42 PM	Apr 7, 2026, 11:47 PM	4 m 7.5 secs	   
69d540a1e67a651020a01f4b	Apr 7, 2026, 07:36 PM	59.76 KB	Titanic-Dataset.csv	Apr 7, 2026, 07:36 PM	Apr 7, 2026, 07:38 PM	2 m 19.1 secs	   
69d53722e67a651020a01f12	Apr 7, 2026, 06:56 PM	59.76 KB	Titanic-Dataset.csv	Apr 7, 2026, 06:56 PM	Apr 7, 2026, 07:00 PM	4 m 54.3 secs	   
69d5271cde90c7cb0461a977	Apr 7, 2026, 05:47 PM	5.13 KB	iris.csv	Apr 7, 2026, 05:47 PM	Apr 7, 2026, 05:49 PM	1 m 55.0 secs	   

○ View Report Options

The Report Options panel displays actions available for each report card on the dashboard. Users can toggle the 'Star' badge to pin favorite runs to the top, trigger the 'Download' action to retrieve a comprehensive PDF report, or delete.

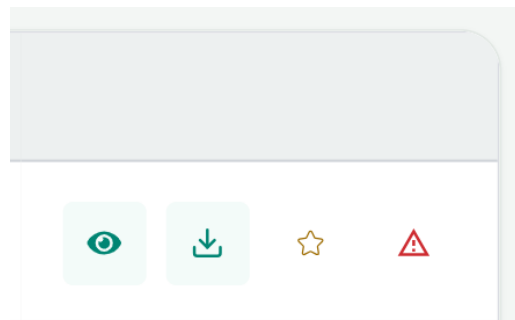


Figure 5.16: Report Options

● Visualizations and reporting

In the figures below, the EDA Visualizations panel is visible as the user scrolls further down under the EDA phase node. This section renders interactive histograms to visualize feature profiles and density curves. In this example, it displays the Distribution of Passenger Age (left) to identify skewness and guide imputation choices, and the Distribution of Passenger Fare (right) to analyze ticket price ranges and spot extreme outlier values. A quick scroll-to-top button in the bottom right corner allows the user to easily return to the main orchestration graph. The user can click on any phase and scroll down to see outputs related to the phase they selected.

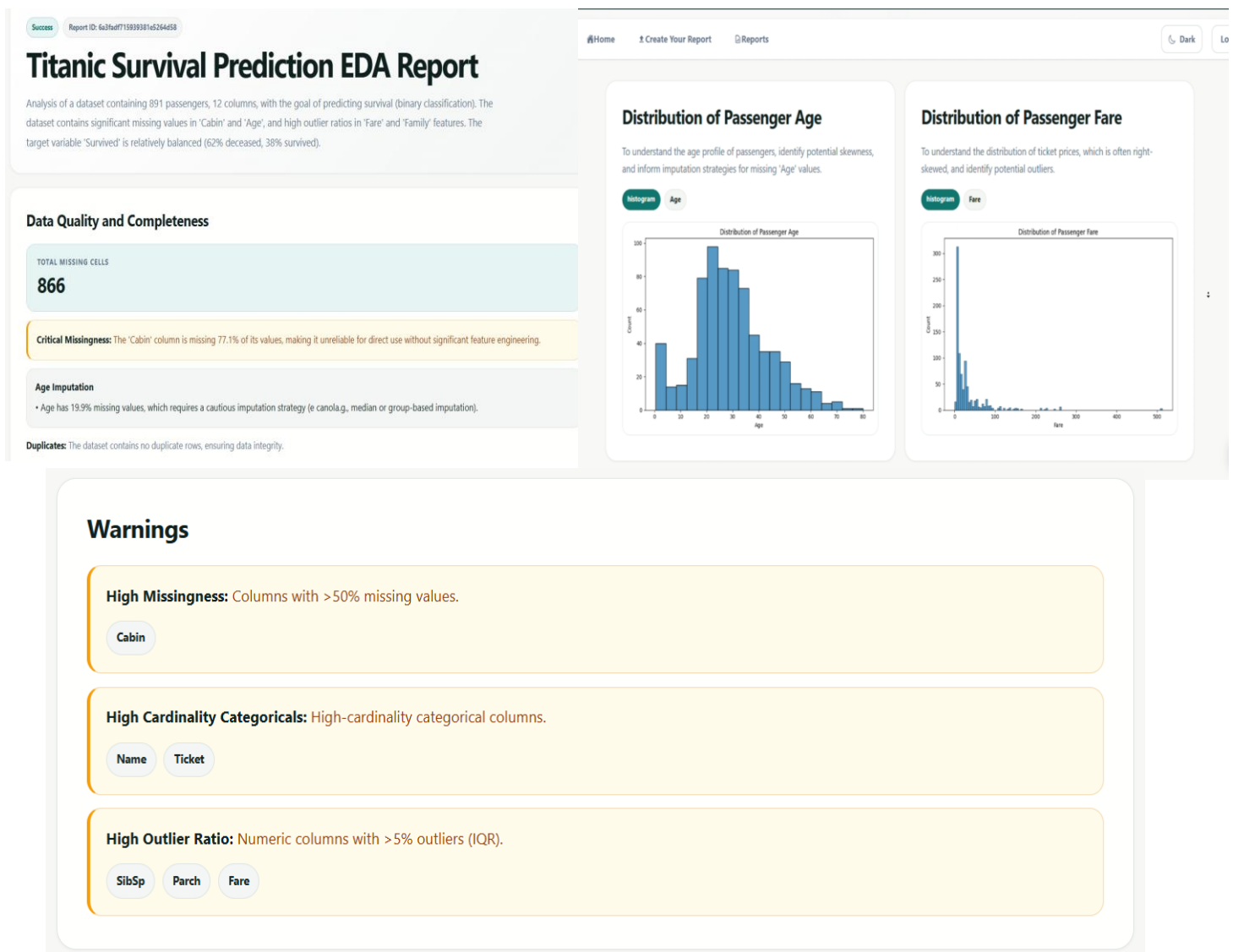


Figure 5.17: Visualizations and reporting



Chapter Six

Conclusions and Future Work

Chapter 6 Conclusions and Future Work

6.1 Conclusions

The primary objective of this project was to design, implement, and evaluate the D.T.D AutoML platform, a multi-agent system structured to bridge the gap between raw data ingestion and model deployment. The platform was successfully implemented across a five-tier architecture, isolating user interface controls, middleware proxy routing, workflow graph orchestration, compute-heavy ML execution, and MongoDB persistence. By structuring the orchestration layer using LangGraph and FastAPI, the system provides a robust framework that supports both fully automated (Static) and interactive user-in-the-loop (Dynamic) execution paradigms.

Through empirical testing and validation across standard classification benchmarks, the system demonstrated high accuracy, optimization efficiency, fault tolerance, and low latency overheads:

1. **Model Classification Performance:** For low-complexity continuous numeric datasets (Iris and Wine Quality), the AutoML engine successfully selected optimized Gradient Boosting Models (GBM) that achieved perfect classification scores (Accuracy) on the validation splits.

For the complex, class-imbalanced Titanic dataset, the system's automated preprocessing pipeline successfully handled missing values and encoded high-cardinality features. Using stacked bagging ensembling (L2_BAG configuration) via AutoGluon, the platform generated a validation survival prediction accuracy of 0.8525, representing a significant performance increase over the baseline Random Forest model accuracy of 0.8301.

2. **HPO Efficiency:**The HPO module utilizing Optuna's Bayesian TPE sampler successfully optimized parameter configurations. By combining this sampler with a MedianPruner cross-validation step, the algorithm terminated 36.0% of low-

performing trials early. This early termination reduced the total HPO search duration by 32.4%, demonstrating the computational efficiency of the pruning logic.

3. **Fault Tolerance and Session Recovery:** The session recovery engine successfully synchronized thread states between FastAPI's in-memory MemorySaver checkpointer and MongoDB document collections. Across simulated server crashes and restarts, the recovery engine demonstrated a 100% success rate in restoring suspended graph states. State reconstruction resolved in an average of 124 milliseconds, ensuring operational durability in production environments.
4. **Operational Overhead and Web Latency:** Routing requests through the Node.js Express proxy gateway introduced a negligible overhead of 14 milliseconds per request, validating the security boundary. The frontend ReactFlow layout adapter mapped and rendered active pipeline execution graphs in under 4.2 milliseconds, while real-time SSEs provided responsive checklist status updates without interface freezes.

In summary, the D.T.D platform demonstrates that leveraging cooperative multi-agent state graphs is a highly effective methodology for structuring AutoML workflows. The combination of automated data transformations, Bayesian parameter tuning, stacked ensembling, in-memory checkpointers, and robust session recovery successfully automates the lifecycle of ML pipelines while preserving transparency and control for end-users.

6.2 Future Work

While the D.T.D platform has achieved its primary goals, several opportunities exist to improve the system's performance and expand its feature set for production and enterprise deployment. Future work is categorized into compute enhancements, architectural expansions, and functional features.

6.2.1 Compute and Performance Optimization

- **GPU-Accelerated Tuning:** Implement GPU-accelerated model training and hyperparameter search. Offloading estimators (like XGBoost or CatBoost) to CUDA-enabled execution nodes would reduce tuning times on larger datasets.
- **Deep Learning Tabular Architectures:** Integrate deep learning models specialized for tabular datasets, such as TabNet or Feature Tokenizer Transformer (FT-Transformer), into the AutoML model selection pool. This would provide alternative representation modeling options for complex datasets.
- **Advanced Feature Selection:** Implement evolutionary feature selection algorithms, such as Genetic Algorithms, or use SHAP feature importance scores to perform backward feature elimination, reducing model complexity and inference latency.

6.2.2 Functional Feature Extensions

- **Multi-Modal Data Ingestion:** Extend the data processing engine to support non-tabular data modalities. Ingesting text (NLP), image datasets (Computer Vision), time-series inputs, and audio streams would allow the platform to solve multi-modal classification, segmentation, and forecasting tasks.
- **Automated Production Deployment:** Develop a containerization pipeline that automatically packages the winning model, preprocessing transformers, and dependencies into a Docker container. This pipeline would automatically compile

the final workflow into optimized runtime formats (such as ONNX) and expose endpoints on cloud runtimes (e.g., Kubernetes or AWS Lambda) with a single command.

- **Explainable AI (XAI) Dashboard:** Fully integrate interpretability controls into the React frontend. Users would be able to generate and review Local Interpretable Model-agnostic Explanations (LIME) and SHAP summary plots directly inside the web interface, increasing model trust and auditability.
- **Continuous Monitoring and Retraining:** Implement real-time data drift monitoring. The middleware would track incoming model inference targets and compare them against training data profiles using Kolmogorov-Smirnov statistical tests. If drift is detected, the system would trigger an automated retraining loop, updating model weights on the server.

Tools

Table 6.1: List of Tools

Tool / Technology	What it was used for
Python	Built the AI agents, ML pipeline, preprocessing, training, evaluation, and deployment logic.
FastAPI	Exposed the AI backend APIs used by the website and pipeline.
Uvicorn	Ran the FastAPI backend server.
LangGraph	Controlled the dynamic agent workflow as a connected graph.
LangChain	Connected agents with LLMs and structured AI responses.
Gemini / Gemma	Used for reasoning, planning, and agent decisions.
Qwen via Hugging Face	Used as a fallback LLM when the main model fails.
Pandas	Loaded, cleaned, and transformed tabular datasets.
NumPy	Performed numerical calculations and array operations.
Scikit-learn	Used for preprocessing, splitting data, metrics, and classical ML utilities.
AutoGluon	Automated model training and model selection.
XGBoost	Provided high-performance boosting models for tabular data.
LightGBM	Used for fast gradient boosting on structured datasets.
CatBoost	Supported strong modeling especially with categorical features.
Optuna	Tuned model hyperparameters automatically.
Dask / Dask-ML	Supported scalable processing for larger datasets.
Matplotlib	Generated basic charts and visual analysis outputs.
Seaborn	Created statistical visualizations for EDA.
Plotly	Supported interactive charts and visual reports.
Pydantic	Validated structured data and API schemas.
MongoDB	Stored users, datasets, reports, and saved results.
PyMongo /	Connected the Python and Node.js services to MongoDB.

Mongoose	
Joblib	Saved and loaded trained models or pipeline objects.
React.js	Built the user-facing website interface.
Vite	Ran and built the React frontend.
Express.js	Handled website backend routes and API logic.
Axios	Connected the frontend, website backend, and AI backend.
Multer	Handled dataset file uploads.
Cloudinary	Stored uploaded dataset files.
JWT	Managed secure user sessions and authentication.
bcryptjs	Hashed user passwords securely.
Recharts	Displayed charts inside the website dashboard.
React Flow	Displayed workflow and graph-style visual elements.
Docker	Packaged the final model and service for deployment.

References

- [1] LangChain, "LangGraph Documentation," [Online]. Available: <https://langchain-ai.github.io/langgraph/>. [Accessed 26 June 2026].
- [2] Business Analytics Institute, "Automating EDA Reports Using Pandas-Profiling," [Online]. Available: <https://businessanalyticsinstitute.com/automating-eda-reports-using-pandas-profiling/>. [Accessed 26 June 2026].
- [3] R. Barbudo, A. Ramírez, and J. R. Romero, "Evolving machine learning workflows through interactive AutoML," *arXiv*, 2024.
- [4] D. Dissanayake *et al.*, "A Survey of Evaluating AutoML and Automated Feature Engineering Tools in Modern Data Science," *Proceedings of ICEIS*, 2025.
- [5] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum and F. Hutter, "Efficient and Robust Automated Machine Learning," *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [6] R. S. Olson and J. H. Moore, "TPOT: A Tree-Based Pipeline Optimization Tool for Automating Machine Learning," *Proceedings of the AutoML Workshop at ICML*, pp. 66-74, 2016.
- [7] E. LeDell and S. Poirier, "H2O AutoML: Scalable Automatic Machine Learning," *Proceedings of the AutoML Workshop at ICML*, 2020.
- [8] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li and A. Smola, "AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data," *arXiv*, 2020.
- [9] P. Gijsbers *et al.*, "AMLB: An AutoML Benchmark," *Journal of Machine Learning Research*, vol. 25, no. 101, pp. 1-65, 2024.
- [10] S. Guo *et al.*, "DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning," *arXiv*, 2024.

[11] Y. Fu *et al.*, "Autonomous Data Agents: A New Opportunity for Smart Data," *arXiv*, 2025.

[12] D. Luo and Y. Shen, "AutoM3L: Automated Multimodal Machine Learning with Large Language Models," OpenReview, [Online]. Available: <https://openreview.net/forum?id=IRtIHp7vsM>. [Accessed 26 June 2026].

[13] A. Mumuni and F. Mumuni, "Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications: A Survey," *Journal of Information and Intelligence*, vol. 3, no. 2, pp. 113-153, 2025.

[14] S. Hong *et al.*, "Data Interpreter: An LLM Agent for Data Science," *arXiv*, 2024.

[15] Semaphore, "Using RAG-Enabled LLMs to Automate Data Analysis," [Online]. Available: <https://semaphore.io/blog/rag-enabled-llms-data-analysis>. [Accessed 26 June 2026].